

Model-Based Reinforcement Learning in Discrete-Action Non-Markovian Reward Decision Processes

Alessandro Trapasso^{1,2}, Luca Iocchi², Fabio Patrizi²

¹Fondazione Bruno Kessler, Trento, Italy

²Sapienza University of Rome, Rome, Italy

atrapasso@fbk.eu, {trapasso, iocchi, patrizi}@diag.uniroma1.it

Abstract

Many practical decision-making problems involve tasks whose success depends on the entire system history, rather than on achieving a state with desired properties. Markovian Reinforcement Learning (RL) can be inadequate for such tasks, while modeling them as non-Markovian reward decision processes (NMRDPs) enables agents to handle temporal dependencies. Existing approaches offer limited formal guarantees on both (near-)optimality and sample efficiency. We address both issues with QR-MAX, a novel model-based algorithm for NMRDPs with discrete actions that factorizes Markovian transition learning from non-Markovian reward handling via reward machines. To our knowledge, this is the first model-based RL algorithm for discrete-action NMRDPs that leverages this factorization to obtain PAC convergence to ϵ -optimal policies with polynomial sample complexity. We then extend QR-MAX to continuous state spaces with BUCKET-QR-MAX, a SimHash-based discretizer that preserves the same factorized structure and achieves fast and stable learning without manual gridding or function approximation. We experimentally compare our method with state-of-the-art model-based RL approaches on environments of increasing complexity, showing substantially improved sample efficiency and greater robustness in finding optimal policies.

1 Introduction

Reinforcement Learning (RL) has achieved striking success in domains that range from game playing to robotics, thanks to its ability to shape behaviour from sparse feedback. Designing such feedback, however, is difficult when the objective spans multiple temporally ordered sub-goals: specifying such tasks with a Markovian reward often yields brittle or misleading signals. Non-Markovian Reward Decision Processes (NMRDPs) address this difficulty by allowing the reward to depend on the environment history; compact logical or automata formalisms, including Reward Machines and LTL_f/LDL_f specifications, allow for encoding these rewards succinctly [Bacchus *et al.*, 1996; Thiébaux *et al.*, 2006; Brafman *et al.*, 2018;

Toro Icarte *et al.*, 2022]. Any NMRDP can be reduced to an ordinary Markov Decision Process (MDP) whose states are pairs (s, q) of environment state and automaton state [Brafman *et al.*, 2018]. Model-free agents operate in $S \times Q$ without storing a transition model [De Giacomo *et al.*, 2020; Toro Icarte *et al.*, 2022], while model-based algorithms such as R-MAX [Brafman and Tenenholz, 2003] can in principle learn a PAC-optimal policy on the same space [Gaon and Brafman, 2020]. Despite this theoretical effectiveness, treating (s, q) as an opaque state discards the fact that the transition kernel is still Markovian and that only the reward component is history-dependent, leading to redundant exploration.

We show that preserving this structural distinction unlocks substantial gains. By factorizing the dynamics into the Markovian environment transition $P(s'|s, a)$ and the deterministic automaton update $q' = \eta(q, s')$, we devise QR-MAX, a model-based algorithm that reuses each learned environment transition for every automaton state and retains separate optimistic bonuses. We show that QR-MAX is PAC-MDP: with high probability, all but a polynomial number of interaction steps are ϵ -optimal. Its sample bound has the factorized unknown-component term $U = |S||A|m_E + |S||Q|m_Q$, where m_E, m_Q are the environment and automaton knowledge thresholds, instead of the product-space count $|S||Q||A|m$ incurred by applying R-MAX directly on $S \times Q$. Thus factorization avoids relearning the environment transition model separately for every automaton state. To the best of our knowledge, this is the first model-based RL algorithm for discrete-action NMRDPs that explicitly leverages the decoupling between environment and automaton dynamics to obtain PAC guarantees with improved sample-complexity bounds. Building on the same idea, we also introduce BUCKET-QR-MAX, a variant of QR-MAX combining SimHash discretisation [Charikar, 2002] with the factorised counts, which extends the approach to continuous state spaces without manual gridding and, under a standard bounded-discretisation assumption, admits a conditional PAC-MDP bound.

We evaluate QR-MAX on five maps, each combined with seven distinct Reward Machines of increasing spatial and temporal complexity, and we also include a continuous-state variant. On the benchmarks we consider, our approach is one to two orders of magnitude more sample-efficient than standard R-MAX, the reward-machine-aware QRM, and optimistic explorers such as UCBVI, PSRL and OPSRL, while

retaining formal PAC guarantees. These results suggest that QR-MAX and its continuous-state extension are practical model-based algorithms with formal sample-complexity bounds for discrete-action NMRDPs. Moreover, optimal discrete-action solutions can drive effective and efficient training in continuous-actions MDPs using hierarchical RL approaches (e.g., [Cipollone *et al.*, 2025]).

2 Related Work

RL with non-Markovian rewards has been studied via NMRDPs, Reward Machines (RMs), and temporal-logic specifications. Two strands are most relevant here: model-based RL for NMRDPs and structure-aware methods for non-Markovian tasks. Prior work applies R-MAX to NMRDPs by incrementally extending an MDP with reward-automaton states, i.e., learning in a representation akin to the R-MAX baseline we use [Gaon and Brafman, 2020]. In contrast, our approach improves sample efficiency by factorizing environment dynamics and reusing learned environment transitions across automaton states.

We also benchmark against state-of-the-art model-based exploration methods for finite MDPs. UCBVI [Azar *et al.*, 2017] matches minimax-optimal regret using empirical counts and confidence bonuses, with memory $\mathcal{O}(S^2A)$. PSRL [Osband *et al.*, 2013] and its optimistic variant OPSRL [Tiapkin *et al.*, 2022] maintain Dirichlet posteriors for transitions, yielding memory $\mathcal{O}(H, S^2A)$ (see Strens [2000], §5.1); this quadratic dependence on S makes them impractical on our largest grids, hence we omit PSRL/OPSRL when the state space is very large. On the 15×15 grid with $H=250$, PSRL/OPSRL need $\approx 50.6\text{M}$ entries vs. 0.204M for QR-MAX, exceeding our budget (see Appendix).

A recent line of work develops structure-aware algorithms for non-Markovian tasks. Exploration in *deterministic* RMs has been studied with regret bounds exploiting automaton structure [Bourel *et al.*, 2023], and extensions to Probabilistic RMs (PRMs) have also been proposed [Bourel *et al.*, 2024; Lin and Zhang, 2025]. These methods typically optimize regret (often average-reward or finite-horizon) under a known RM/PRM, and are therefore not directly comparable to our discounted PAC-style evaluation; in our setting we provide a tabular model-based baseline and improve over R-MAX on $S \times Q$ by reusing environment transitions across automaton states. Our factorization is orthogonal to factored MDPs, where the state is represented by variables and the transition model factorises as a dynamic Bayesian network [Kearns and Koller, 1999; Guestrin *et al.*, 2003; Strehl, 2007; Osband and Roy, 2014]. Here, the underlying environment need not be factored: the gain comes from separating Markovian environment dynamics from the non-Markovian reward automaton. Other NMRDP and temporal-logic approaches either estimate alternative dynamics or focus mostly on model-free learning [Gupta *et al.*, 2021; Toro Icarte *et al.*, 2022; De Giacomo *et al.*, 2020], whereas we focus on model-based PAC learning. Among model-free methods, one approach maximizes the probability of satisfying an LTL formula [Shao and Kwiatkowska, 2023]; it differs from our setting in focusing on infinite traces, while we use

finite-trace automata (regular languages). Nonetheless, on finite traces such approaches reduce to Q-learning and QRM, which we compare against, so our experiments indirectly account for [Shao and Kwiatkowska, 2023].

Finally, a substantial body of work aims to learn the NMR model itself, e.g., [Gaon and Brafman, 2020]; this is outside our scope since we assume a given NMR specification. To the best of our knowledge, QR-MAX is the first tabular model-based RL algorithm for discounted NMRDPs that factorizes environment and reward dynamics to obtain PAC-MDP guarantees with improved sample-complexity bounds over R-MAX on $S \times Q$.

3 Background

A *Markov Decision Process* (MDP) is a tuple $\mathcal{M} = (S, A, P, R_E)$ with finite states S , actions A , transition kernel $P(s'|s, a)$, and reward function $R_E : S \times A \times S \rightarrow \mathbb{R}$. A *policy* $\rho : S \rightarrow A$ induces the *return* $v^\rho(s) = E_\rho[r_0 + \gamma r_1 + \gamma^2 r_2 + \dots]$, where $r_i = R_E(s_i, \rho(s_i), s_{i+1})$, $s_0 = s$, and $\gamma \in [0, 1]$ is the *discount factor*. The goal is to find an *optimal policy* ρ^* maximizing $v^*(s)$ for all $s \in S$. In RL, the functions P and R_E are unknown and must be learned through interaction.

RL methods are typically *model-free* or *model-based*: the former seek ρ^* without explicitly estimating P and R_E , while the latter learn both. Two representative algorithms are Q-LEARNING [Watkins and Dayan, 1992] and R-MAX [Brafman and Tennenholtz, 2003]. A central notion is the optimal state-action value function $Q^* : S \times A \rightarrow \mathbb{R}$, where $Q^*(s, a)$ is the expected discounted return obtained by executing a in s and then acting optimally; an optimal policy is $\rho^*(s) \in \arg \max_a Q^*(s, a)$. While Q^* can be computed from P and R_E (e.g., via *Value Iteration* [Sutton and Barto, 2018]), this is not directly possible in RL since P and R_E are unknown.

Q-LEARNING maintains an estimate $\hat{Q}$, initialized arbitrarily and updated from experience using an update rule based on *Bellman’s optimality condition*; with sufficient exploration, $\hat{Q}$ approximates Q^* and induces a near-optimal policy. In contrast, R-MAX estimates P and R_E by collecting samples for state-action pairs (s, a) until they are deemed *known*; it then plans in the learned model (typically via value iteration), using optimism to drive exploration. In standard RL, R_E is *Markovian*, depending only on the current (and possibly next) state; here we consider *non-Markovian* rewards.

A *Non-Markovian Reward Decision Process* (NMRDP) is a tuple $\mathcal{N} = (\mathcal{M}, R_A)$, where $R_A : S^+ \rightarrow \mathbb{R}$ is a *Non-Markovian Reward Function* (NMR) depending on state histories. NMRs can be specified compactly, e.g., by Reward Machines [Toro Icarte *et al.*, 2022] or Restraining Bolts [De Giacomo *et al.*, 2020]. We unify them using a *Deterministic Finite-state Automaton* (DFA) $\mathcal{A} = (S, Q, q_0, \eta, F)$, where S is the input alphabet, Q the automaton states, q_0 the initial state, $\eta : Q \times S \rightarrow Q$ the transition function, and F the final states. We assume full observability of both the environment state and the automaton state: after executing an action, the agent observes (s', q') (equivalently, it can update q' deterministically when $\mathcal{A}$ is known). The reward is given by $R_A(q, s', q')$ for the last automaton transition (q, s', q') . In

this notation, a reward machine is a deterministic automaton endowed with transition rewards, while a restraining bolt is the DFA induced by the temporal specification; acceptance can be encoded through R_A , e.g., by assigning positive reward to transitions entering F . Thus F has no separate algorithmic role unless acceptance itself is part of the reward specification.

$\mathcal{A}$ compactly captures temporal properties of the history; in practice $|Q| \ll |S|$. In an NMRDP $\mathcal{N} = (\mathcal{M}, R_A)$ with $\mathcal{M} = (S, A, P, R_E)$, after a trajectory $s_0 a_0 \dots s a s'$, the reward is $R_E(s, a, s') + R_A(q, s', q')$, where q (resp. q') is the DFA state after consuming $s_0 \dots s$ (resp. $s_0 \dots s s'$). Although this makes the reward *stateful*, NMRDPs can be reduced to Markovian RL by augmenting the state with the automaton component. As shown in [Brafman *et al.*, 2018], $\mathcal{N}$ is equivalent to an MDP $\mathcal{M}' = (S', A, P', R')$ over $S' = S \times Q$, with P' capturing synchronous transitions and R' Markovian. Standard RL methods can then be applied (see, e.g., [De Giacomo *et al.*, 2020; Toro Icarte *et al.*, 2022]).

However, this reduction treats (s, q) as an atomic state, losing the distinction between the MDP dynamics ($\mathcal{M}$) and the DFA dynamics ($\mathcal{A}$). For instance, learning transitions from (s, q) on action a does not help with (s, q') , even though the environment dynamics from s are identical. Since $\mathcal{A}$ is deterministic (and typically easier to learn), this coupling can yield redundant exploration. We investigate how to retain this opportunity by factorizing $\mathcal{N}$'s dynamics into $\mathcal{M}$'s and $\mathcal{A}$'s.

4 Decoupling Environment and Reward Dynamics

We model an NMRDP $\mathcal{N} = (\mathcal{M}, R_A)$, by combining the dynamics of the environment $\mathcal{M}$ with that of the NMR R_A and the DFA $\mathcal{A}$ it is based on, as follows: (1) when the system is reset, the environment is set to an initial state s_0 , $\mathcal{A}$ is set to its initial state q_0 , and a dummy DFA transition $q_0 = \eta(q_0, s_0)$ is performed, s.t. $\mathcal{A}$ consumes s_0 ; (2) at each step, given the current state (s, q) and the chosen action a , the environment progresses to a state s' (with probability $P(s'|s, a)$), $\mathcal{A}$ progresses to $q' = \eta(q, s')$ (with probability 1), and the returned total reward is $R_E(s, a, s') + R_A(q, s', q')$.

The resulting transition function is captured as follows:

$$P(s', q'|s, q, a) = P(q'|s', q, s, a) P(s'|s, q, a),$$

where $P(s', q'|s, q, a)$, $P(q'|s', q, s, a)$, $P(s'|s, q, a)$ are the transition probability distributions of $\mathcal{N}$, R_A 's automaton $\mathcal{A}$, and the environment $\mathcal{M}$, respectively. However, since $\mathcal{M}$ is Markovian: $P(s'|s, q, a) = P(s'|s, a)$, i.e., given s and a , s' is independent of the history leading $\mathcal{A}$ to q . Also, since q' depends only on q and s' : $P(q'|s', q, s, a) = P(q'|s', q)$. Thus, $\mathcal{N}$'s transition model is:

$$P(s', q'|s, q, a) = P(s'|s, a) P(q'|s', q). \quad (1)$$

As for the reward function, we simply have:

$$R(s, q, a, s', q') = R_E(s, a, s') + R_A(q, s', q'). \quad (2)$$

Note that $R_A(q, s', q')$ does not depend on s or a , i.e., on how state (s', q') has been reached from any state (s, q) .

Equations 1 and 2 describe a factorization of the system dynamics that decouples the Markovian environment dynamics ($P(s'|s, a)$, $R_E(s, a, s')$) from the non-Markovian reward expressed by the DFA ($P(q'|s', q)$, $R_A(q, s', q')$).

Observability and RM Knowledge. We assume the agent observes the current automaton state q (as standard in RM settings via a monitor over observable propositions) and receives the RM reward component r_A from the RM monitor/wrapper. We consider two settings: QR-MAX does *not* assume knowledge of the automaton transition function η (it only receives the $q \rightarrow q'$ signal), while QR-MAXRM assumes the RM is given (including η), enabling counterfactual updates across automaton states. If q is not observable, the problem becomes partially observable and is outside our scope.

Learning Algorithms. Discrete model-based RL can be applied in the joint state space $S \times Q$ to estimate the transition and reward functions $P(s', q'|s, q, a)$ and $R(s, q, a, s', q')$. In this paper, we compare our method based on the above factorization with different model-based RL approaches that use the joint space $S \times Q$, such as that in [Brafman *et al.*, 2018; Gaon and Brafman, 2020]. As shown in the experiments, the use of the joint space does not scale well with the task's complexity, since it does not exploit the Markovian nature of the environment transitions.

4.1 QR-MAX Algorithm

In this section, we describe QR-MAX, a novel algorithm extending R-MAX to non-Markovian rewards and Markovian transitions, by exploiting the factorization described above. As shown later on, QR-MAX provides the same guarantees for near-optimal solutions as R-MAX, but exhibits a significantly higher sample efficiency. The main advantage of QR-MAX is that the terms of Equations 1 and 2, i.e., $P(s'|s, a)$, $P(q'|s', q)$, $R_E(s, a, s')$, and $R_A(q, s', q')$, can be estimated in parallel by setting different thresholds, t_E and t_Q , to determine when they are *known*. For $n_E(s, a)$ and $n_Q(q, s')$ the number of visits to the environment's and DFA's states, respectively, we use the following conditions to label states as known: $n_E(s, a) \geq t_E \implies \text{Known}(s, a)$ and $n_Q(q, s') \geq t_Q \implies \text{Known}(q, s')$.

The experience for $n_E(s, a)$ is collected independently of the state of $\mathcal{A}$ and once $\text{Known}(s, a)$ holds and the environment transition and reward functions are estimated, respectively as $\hat{P}(s'|s, a)$ and $\hat{R}_E(s, a, s')$, these values are used also in other unseen states of $\mathcal{A}$. This decoupling yields a significant improvement in sample efficiency with a standard, monolithic formulation of the joint space. Observe, in particular, that for deterministic $\mathcal{A}$ (as is the case here), $t_Q = 1$ is sufficient to know all $\mathcal{A}$'s states, thus yielding a significant saving in time for learning $\mathcal{A}$. We keep the threshold t_Q for modularity and generality, to make the algorithm applicable also to more general machines, such as *stochastic* RMs [Corazza *et al.*, 2022].

Following R-MAX's approach, QR-MAX (Algorithm 1) initializes the Q-table QT with a maximum value (line 1) for every state-action pair (notice states now consist of s and q) and selects the next action to execute as the argmax of the current Q-table (line 8). In lines 11–26, environment transitions and rewards (τ_E, ρ_E, n_E) and automaton transitions and

Algorithm 1. QR-MAX

Input: S, Q, A states and actions, E environment,
 γ discount factor, $R_{\max}$ max expected reward,
 t_E, t_Q thresholds, T value iteration horizon
Output: QT optimal Q-table

```

 $QT(s, q, a) \leftarrow R_{\max}/(1-\gamma), \forall s \in S, q \in Q, a \in A$ 
 $\tau_E(s, a, s') \leftarrow 0, \rho_E(s, a, s') \leftarrow 0, \forall s, s' \in S, a \in A$ 
 $n_E(s, a) \leftarrow 0, \forall s \in S, a \in A$ 
 $\tau_Q(q, s', q') \leftarrow 0, \rho_Q(q, s', q') \leftarrow 0, \forall s' \in S, q, q' \in Q$ 
 $n_Q(q, s') \leftarrow 0, \forall s' \in S, q \in Q$ 
 $s, q \leftarrow E.reset()$ 
for  $t = 1, \dots, N_{steps}$  do
   $\bar{a} \leftarrow \arg\max_a QT(s, q, a)$ 
   $s', q', r_E, r_A, done = E.step(\bar{a})$ 
   $doVI = \text{False}$ 
  if  $n_E(s, \bar{a}) < t_E$  then
     $\tau_E(s, \bar{a}, s') += 1$ 
     $\rho_E(s, \bar{a}, s') += r_E$ 
     $n_E(s, \bar{a}) += 1$ 
    if  $n_E(s, \bar{a}) = t_E$  then
       $doVI = \text{True}$ 
  if  $n_Q(q, s') < t_Q$  then
     $\tau_Q(q, s', q') += 1$ 
     $\rho_Q(q, s', q') += r_A$ 
     $n_Q(q, s') += 1$ 
    if  $n_Q(q, s') = t_Q$  then
       $doVI = \text{True}$ 
  if  $done$  then
     $QT(s', q', a') \leftarrow 0, \forall a' \in A$ 
  if  $doVI$  then
     $QT \leftarrow VI(QT, \tau_E, \rho_E, n_E, \tau_Q, \rho_Q, n_Q, \gamma, T)$ 
  if  $done$  then
     $s, q \leftarrow E.reset()$ 
  else
     $s, q \leftarrow s', q'$ 
return  $QT$ 

```

rewards (τ_Q, ρ_Q, n_Q) are accumulated separately, using different thresholds, t_E, t_Q . If any pair (s, a) or (q, s') reaches the corresponding target threshold (lines 15-17 and 23-25), a value iteration step (VI) is run for T iterations (line 31) to update the Q-table values. When an episode terminates, the Q-table values for the final states are set to zero (line 28) and the environment is reset (line 34). For value iteration, a standard algorithm is adopted, which uses Equations 1 and 2 to describe the process and estimate the model functions as follows, considering only *known* pairs:

$$\hat{P}(s'|s, a) = \frac{\tau_E(s, a, s')}{n_E(s, a)}, \quad \hat{P}(q'|s', q) = \frac{\tau_Q(q, s', q')}{n_Q(q, s')},$$

$$\hat{R}_E(s, a, s') = \frac{\rho_E(s, a, s')}{n_E(s, a)}, \quad \hat{R}_Q(q, s', q') = \frac{\rho_Q(q, s', q')}{n_Q(q, s')}.$$

Bellman Backup Used in VI. For known components, VI:

$$Q_{k+1}(s, q, a) = \sum_{s'} \hat{R}_E(s, a, s') + \sum_{s'} \hat{P}(s'|s, a) \left(\sum_{q'} \hat{R}_Q(q, s', q') + \gamma \sum_{q'} \hat{P}(q'|s', q) \max_{a'} Q_k(s', q', a') \right). \quad (3)$$

Here $\hat{R}_E(s, a, s')$ and $\hat{R}_Q(q, s', q')$ are *reward-mass* estimates: $\hat{R}_E(s, a, s') = \hat{P}(s'|s, a) \hat{r}_E(s, a, s')$ and $\hat{R}_Q(q, s', q') = \hat{P}(q'|s', q) \hat{r}_Q(q, s', q')$. Thus Eq. (3) is identical to the canonical expected-reward backup, but written in mass form to avoid conditioning on rare successor events.

During value iteration we update $QT(s, q, a)$ only when $\text{KNOWN}_E(s, a)$ holds and, for every successor state s' with

$\tau_E(s, a, s') > 0$, also $\text{KNOWN}_Q(q, s')$ holds. All other entries retain their initial optimistic value $R_{\max}/(1-\gamma)$; equivalently, any unobserved successor probability mass is assigned to an absorbing optimistic state with value $R_{\max}/(1-\gamma)$ (standard R-MAX), underpinning the PAC-MDP analysis in Section 4.2.

In contrast to RM-aware methods that assume direct access to the automaton transition function η , QR-MAX does not require η as an input: it only assumes access to the automaton-state signal, i.e., the agent observes the current automaton state q and, after each action, the updated state q' together with the reward, as formalized above. If q is not observable and the RM is unknown, the problem becomes partially observable; we leave automaton-state inference to future work.

When the DFA/RM is available, we can additionally generate counterfactual experience across automaton states and apply the same factorized updates to it, obtaining the structure-aware variant QR-MAXRM, which we also evaluate experimentally.

4.2 Theoretical Guarantees of QR-MAX

Let $X = S \times Q$, $V_{\max} = R_{\max}/(1-\gamma)$, and $U = |S||A|m_E + |S||Q|m_Q$, where U is the number of factorized components that can be unknown before becoming known. We assume one-step rewards are bounded in $[0, R_{\max}]$. For the PAC analysis, we denote the knownness thresholds by $m_E = t_E$ and $m_Q = t_Q$. The thresholds m_E, m_Q are chosen by standard concentration bounds and a union bound over the events for R_E, P_E, R_A , and P_Q , so that, with probability at least $1 - \delta$, every known component induces reward error at most ξ_R , environment-transition error at most ξ_E , and automaton-transition error at most ξ_Q . They are chosen so that $2(\xi_R + \gamma V_{\max}(\xi_E + \xi_Q))/(1-\gamma) + \eta_{VI} \leq \epsilon$, where η_{VI} is the value-iteration truncation error. For deterministic reward machines with deterministic transition rewards, one observation suffices for each observed (q, s') , hence $m_Q = 1$ and $\xi_Q = 0$ on known automaton components.

Lemma 1 (Factored Accuracy). *On the above event, every known product component satisfies $|\hat{R} - R| \leq \xi_R$ and $\|\hat{P}(\cdot | s, q, a) - P(\cdot | s, q, a)\|_1 \leq \xi_E + \xi_Q$. Indeed, from Eq. (1), $\|\hat{P} - P\|_1 \leq \|\hat{P}_E(\cdot | s, a) - P_E(\cdot | s, a)\|_1 + \sum_{s'} \hat{P}_E(s' | s, a) \|\hat{P}_Q(\cdot | q, s') - P_Q(\cdot | q, s')\|_1 \leq \xi_E + \xi_Q$.*

Lemma 2 (Near-Optimality in the Known Model). *Conditioned on Lemma 1, the greedy policy computed in the optimistic model is ϵ -optimal whenever no unknown component is reached within the effective horizon. This follows from the discounted simulation bound $\|\hat{T}V - TV\|_{\infty} \leq \xi_R + \gamma V_{\max}(\xi_E + \xi_Q)$, valid for all $\|V\|_{\infty} \leq V_{\max}$, and from the above choice of ξ_R, ξ_E, ξ_Q , and η_{VI} .*

Theorem 3 (PAC-MDP Bound). *With the above thresholds, QR-MAX is PAC-MDP. With probability at least $1 - \delta$, the number of interaction steps in which its greedy policy is not ϵ -optimal is bounded by the standard optimistic-model escape bound $O((HU/p) \log(U/\delta))$, where $H = \lceil (1-\gamma)^{-1} \ln(2R_{\max}/[\epsilon(1-\gamma)]) \rceil$, $p = \epsilon(1-\gamma)/(2R_{\max})$, and $U = |S||A|m_E + |S||Q|m_Q$. Equivalently, the leading unknown-component dependence is factorized rather than product-based.*

Proof Sketch. Condition on the event of Lemma 1. If the optimistic greedy policy is not ϵ -optimal, Lemma 2 implies that, by the standard R-MAX escape-event argument [Brafman and Tennenholtz, 2003; Strehl *et al.*, 2009], it must reach an unknown component within horizon H with probability at least p . Each escape is charged to the first unknown component reached. Since environment components can be charged at most $|S||A|m_E$ times and automaton components at most $|S||Q|m_Q$ times, the standard PAC-MDP argument yields the stated bound. Value iteration affects computation time but does not consume environment interactions.

Product-Space R-MAX Comparison. Applied directly to the product MDP over $S \times Q$, R-MAX learns unknown components indexed by (s, q, a) , giving a product-space count of order $|S||Q||A|m$. In contrast, QR-MAX learns the environment component once for each (s, a) and the automaton component once for each (q, s') , giving the factorized count $U = |S||A|m_E + |S||Q|m_Q$. Thus the expensive environment-transition estimation is not repeated for every automaton state. In deterministic reward machines, where $m_Q = 1$, the dominant learned component is typically the shared environment model, which gives the main sample-efficiency advantage over product-space R-MAX.

A complete proof is provided in Appendix A.

5 Bucket-QR-Max: Continuous State Spaces

QR-MAX assumes that the environment state component $s \in S$ is *discrete*. In many real-world problems the agent instead observes a continuous vector $s^{\text{cont}} \in \mathbb{R}^d$, while actions remain finite and discrete. A common way to handle continuous states is to discretise them into a finite set of “buckets” and treat each bucket as a discrete state. We follow this idea and integrate a hash-based discretisation into QR-MAX, yielding BUCKET-QR-MAX, a model-based extension that retains the structural advantages of QR-MAX (separate counts for Markovian dynamics and non-Markovian reward, optimistic value iteration) while handling continuous observations *without neural approximation*. Under a bounded-discretisation assumption on the induced buckets (formalised as Assumption B.1 in Appendix B), we derive a conditional PAC-MDP bound for BUCKET-QR-MAX in the underlying continuous MDP (Theorem B.2).

Locality-Sensitive Hashing Discretisation

Let $h : \mathbb{R}^d \rightarrow \mathcal{B}$ be a family of L independent SimHash functions [Charikar, 2002], $h(\cdot) = (h_1(\cdot), \dots, h_L(\cdot))$, where each h_ℓ projects s^{cont} onto $\{0, \dots, 2^{d_h} - 1\}$. Given s_t^{cont} we obtain a bucket $b_t = h(s_t^{\text{cont}}) \in \mathcal{B} \subset \mathbb{N}^L$. The mapping is *data-dependent*: new buckets are added on-the-fly and assigned the next integer index $\iota : \mathcal{B} \rightarrow \{0, \dots, B_t\}$. A joint observation is therefore represented as $(b, q) \in \mathcal{B} \times Q$. The automaton component remains exact and finite, while the continuous part is covered by a dynamically growing *countable* partition.

Bucket-QR-Max Counters and Reward Sums

It keeps $n_{E_T}(b, a)$ for environment transitions, $n_{E_R}(b, a)$ and $R_E(b, a)$ for environment rewards, $n_{Q_T}(q, b')$

for reward-machine transitions, and $n_{Q_R}(q, b')$ and $R_Q(q, b')$ for non-Markovian rewards. The corresponding known conditions are $(n_{E_T}(b, a), n_{E_R}(b, a)) \geq (t_{E_T}, t_{E_R})$, $(n_{Q_T}(q, b'), n_{Q_R}(q, b')) \geq (t_{Q_T}, t_{Q_R})$. Only when *all four* conditions are met is the triple (b, a, q) marked *known* and Value Iteration triggered. The thresholds $t_{E_T}, t_{E_R}, t_{Q_T}, t_{Q_R}$ are chosen by the same PAC-MDP concentration argument as in Sec. 4.2, applied to the induced bucket MDP. The agent maintains an *optimistic* Q-table $Q[b, q, a] = R_{\max}/(1 - \gamma)$ for every discovered bucket b and runs synchronous Value Iteration over the finite abstraction on $\iota(\mathcal{B}) \times Q$ until convergence up to ε_{VI} . The tabular solver therefore keeps QR-MAX’s factorised counts and an optimistic value function over $\mathcal{O}(|\mathcal{B}_t||Q||A|)$ entries.

Exploration and Optimism

Because the hash granularity is fixed *a priori*, each bucket b plays the role of a discrete state s in QR-MAX: the same factorised counters and model estimates (P_E, P_Q, R_E, R_Q) are reused, and the Q-table remains optimistic at $R_{\max}/(1 - \gamma)$ until (b, a, q) becomes known. Thus BUCKET-QR-MAX inherits exactly the same optimistic-exploration mechanism as the discrete algorithm.

Design Intuition and Conditional PAC Guarantee

For any fixed finite partition $\mathcal{B}$ of the state space, BUCKET-QR-MAX reduces to running QR-MAX on the abstract MDP with states $(b, q) \in \mathcal{B} \times Q$. In Appendix B, we formalise a bounded-discretisation assumption (small bucket diameter and Lipschitz-continuous dynamics and rewards, see Assumption B.1) and show that, under this assumption, the induced bucket MDP is a close approximation of the underlying continuous MDP. The resulting conditional PAC-MDP bound (Theorem B.2) mirrors the discrete bound in Theorem 3, with $|S|$ replaced by $|\mathcal{B}|$. Intuitively, when nearby continuous states tend to fall into the same bucket and the environment is sufficiently smooth, an ε -optimal policy for the bucket MDP is also near-optimal in the original continuous MDP. In practice, SimHash does not deterministically bound bucket diameters; our PAC guarantee is conditional on the induced partition satisfying the bounded-diameter assumption, and hash collisions can be absorbed into an additional failure probability δ_{LSH} (overall success at least $1 - (\delta + \delta_{\text{LSH}})$), see Appendix B for details.

Relation to Bucket-R-Max and Implementation.

The *Covering-RMAX* algorithm replaces exact states in R-MAX with a balls-in-metric-space partition [Bernstein and Shimkin, 2008]. BUCKET-QR-MAX differs in two essential ways:

1. **Hash vs. metric balls.** LSH only probabilistically groups nearby states: by increasing the number of hash functions and projections, the probability that distant states collide in the same bucket can be made small. SimHash needs *no* explicit metric and updates in $O(L)$ time, whereas searching the nearest-centre set grows with the cover size. In high-dimensional spaces, Covering-RMAX can suffer from the curse of dimensionality, as the number of metric balls required grows with the dimension, whereas for a fixed hash family SimHash maintains

a fixed per-update computational cost.

2. **Decoupled NM-reward.** We preserve the QR-MAX factorization, yielding a sample complexity in $|\mathcal{B}||A| + |\mathcal{B}||Q|$ instead of the cubic $|\mathcal{B}||Q||A|$ inherited by a naive continuous R-MAX reduction.

BUCKET-QR-MAX differs from discrete QR-MAX in exactly three lines: (i) *LSH discretisation* $b \leftarrow h(s^{\text{cont}})$; (ii) four separate counters $n_{E_T}, n_{E_R}, n_{Q_T}, n_{Q_R}$, each with its own threshold; only when all four exceed their thresholds is (b, a, q) marked known and VI invoked; (iii) all counters, model estimates, and Q-values are indexed by buckets b instead of discrete states s . Full pseudocode is reported in Algorithm 2 in Appendix B.

6 Experiments

In this section, we compare QR-MAX and QR-MAXRM against several state-of-the-art algorithms, on different environments of increasing difficulty.¹ Our goal is to show that the factorization introduced in QR-MAX enhances sample efficiency and enables optimal solution of complex tasks, confirming the theoretical results obtained in Sec. 4.

The compared methods include: 1) model-based methods for MDPs over the joint space $S \times Q$: UCBVI-sB (simplified Bernstein bonus), UCBVI-H (Hoeffding bonus), and UCBVI-B (Bernstein bonus) from [Azar *et al.*, 2017]; PSRL [Osband *et al.*, 2013]; OPSRL [Tiapkin *et al.*, 2022]; R-MAX [Brafman and Tennenholtz, 2003]; 2) the model-free method QRM [Toro Icarte *et al.*, 2022] for NMRDP. We do not compare directly with [Gaon and Brafman, 2020], which focuses on learning the RM, since when this is available, the approach corresponds to R-MAX, which we evaluate. Also, as already mentioned, [Shao and Kwiatkowska, 2023] is indirectly compared, corresponding, on the problems considered, to QRM. For clarity, QR-MAX uses only the observed automaton-state signal (q) and does not assume access to η , whereas QR-MAXRM assumes the RM is given (including η) and uses it for counterfactual updates across automaton states.

Unless stated otherwise, we use the authors’ recommended hyper-parameters. For finite-horizon baselines that require a horizon (UCBVI, PSRL/OPSRL), we set H to the VI-optimal path length plus a small safety margin (Appendix C); this uses oracle knowledge to choose a favorable H for these baselines, hence the comparison is conservative w.r.t. QR-MAX. The benchmark we use for experimental comparison takes inspiration from, and extends, the Office gridworld environment [Toro Icarte *et al.*, 2022] (full details are reported in Appendix C), used to evaluate NMRDP solutions. In all the experiments, during training, we use stochastic actions, all having a nominal outcome, i.e., one with highest probability wrt all the others.

For each experimental configuration, we compute a VI baseline by running value iteration on the full MDP model (obtained from the environment). Periodically we compare the current learned policy (greedy w.r.t. the Q-table) to the VI policy by running 10,000 evaluation episodes per policy in the stochastic environment and collecting discounted episode returns. We apply a two-sided Welch t-test to the two

return samples. With early stopping enabled, in stochastic environments training terminates when the Welch t-test fails to detect a significant difference at a 10% significance level ($p\text{-value} \geq 0.1$), treating the two policies as statistically indistinguishable. In deterministic settings the t-test is skipped due to zero variance. We apply this rule uniformly; additional Appendix curves (e.g., fixed-budget success rates) confirm the same ordering. For visualization purposes, we run the learned policies on the corresponding deterministic testing environments, where actions always produce their nominal outcome. This simplifies comparing the learned policies with VI solutions, in terms of required timesteps to reach the goal. We report mean $\pm$ std over 10 seeds on three configurations.

Configuration 1 — map0_exp0 (10×10). We use the simple grid world from prior work [Tiapkin *et al.*, 2022], using $H = 50$ and eight Thompson samples, following the original papers. The agent’s non-Markovian task consists in starting from the top-left cell, collecting a letter in the bottom-right cell, returning to the start position to pick up a coffee, and finally delivering both items to the office in the bottom-left cell. The issued reward is 1 on task completion, 0 otherwise. Figure 1 shows the success rate: QR-MAXRM and QR-MAX achieve significantly better performance in roughly one order of magnitude fewer training steps than UCBVI, PSRL, and OPSRL, confirming that the factorised dynamics improves sample efficiency.

Configuration 2 — map1_exp5 (Office World, 12×9). This map contains walls and ornamental objects that impose a penalty of -100 upon contact. For UCBVI and its variants we set $H = 150$. As illustrated in Figure 2, as task complexity grows, QR-MAX and QR-MAXRM maintain a gap of nearly two orders of magnitude over UCBVI-H and UCBVI-B, and about one order over QRM (which has access to the RM), R-MAX, and UCBVI-sB. PSRL and OPSRL exceed the predefined memory/CPU budget on this instance (see Appendix C for the resource limits).

Configuration 3 — map4_exp6 (15×15). The third configuration is a maze-like space featuring two coffee machines: a *good* one (reward 1000) and a *regular* one (reward 1). The agent receives a reward only after picking up a coffee and delivering it to the office. Figure 3 shows that QR-MAX and QR-MAXRM converge to the optimal policy, bringing the better coffee in 57 steps, whereas QRM, UCBVI and their variants remain trapped for many training steps in the local optimum associated with the nearer coffee machine.

Evaluation of BUCKET-QR-MAX. We evaluate BUCKET-QR-MAX on a continuous-state variant of Frozen Lake and Office World (5 subtasks), using SimHash-based discretizations and the same factorized counting scheme as in the discrete case (see Appendix for details). As baselines, we train a DQN on the same inputs (s^{cont}, q) and sparse non-Markovian reward, with a standard two-layer MLP and replay buffer, and we also include a controlled R-MAX variant that uses the same SimHash buckets but learns a monolithic model over joint states (b, q) (see Appendix). In this setting BUCKET-QR-MAX reaches near-optimal performance in 50 ± 10 episodes, over an order of magnitude faster than DQN (which requires > 500

¹Code: <https://github.com/Alee08/qrmx>.

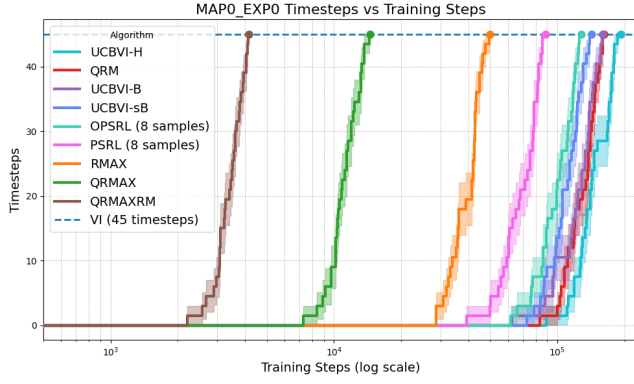


Figure 1: Configuration 1 (map0 exp0).

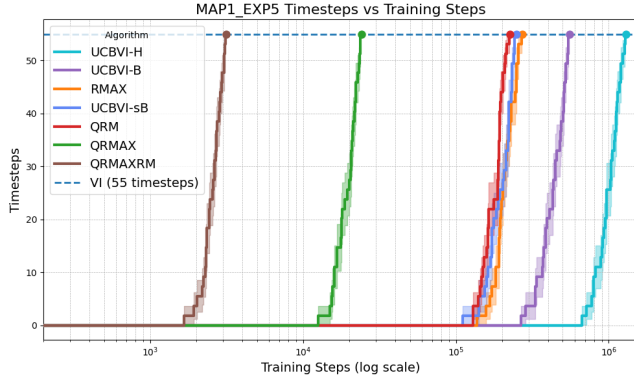


Figure 2: Configuration 2 (map1 exp5).

episodes) and with lower variance, indicating that combining SimHash discretizations with separate environment and automaton counters yields fast, stable learning in continuous state spaces without manual gridding or neural function approximation.

Summary of the Results. The experimental analysis demonstrates that, by exploiting model-based RL and the proposed factorization, QR-MAX, QR-MAXRM, and BUCKET-QR-MAX consistently achieve better sample efficiency than both model-free baselines and model-based counterparts without factorization on all the scenarios we consider. Wall-clock times remained comparable to UCBVI within the same resource limits (see Appendix). We also observe different impacts of hyper-parameter tuning on solution quality. The main hyper-parameters of QR-MAX and variants are the thresholds t_E and t_Q , which control the number of samples needed to reliably estimate transitions and rewards. We set $t_E = 39$ empirically and $t_Q = 1$ because the considered RMs are deterministic. In practice, t_E is relatively easy to set because performance is weakly sensitive beyond the threshold, and training steps scale roughly linearly with t_E . In contrast, the hyper-parameters for the other methods are significantly more difficult to tune and yield higher result sensitivity. For instance, model-free RL algorithms require setting the learning rate, exploration factors, and initial Q -table values, resulting in high impact on the solutions; moreover, such parameters cannot easily be related to the specific problem. The

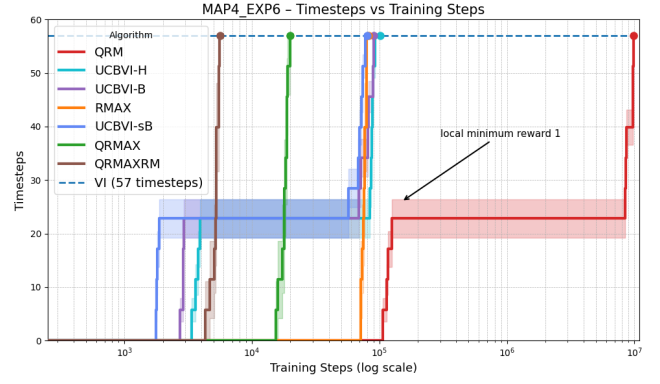


Figure 3: Configuration 3 (map4 exp6).

Map	Exp	QR-MAX	QR-MAXRM	R-MAX	QRM	UCBVI-sB	UCBVI-B	UCBVI-H	PSRL/OPSRL [†]
MAP0	EXP0	14 483	4 150	49 759	162 040	142 015	159 092	191 983	88 437 / 128 188
MAP1	EXP5	24 222	3 125	272 080	225 140	250 800	555 977	1 320 070	—
MAP4	EXP6	20 150	5 630	82 000	9 910 340	80 160	90 030	101 120	—

[†]Eight posterior samples per episode, as in prior work [Tiapkin *et al.*, 2022].

Table 1: Steps to a VI-optimal policy (mean over 10 seeds).

other model-based RL algorithms considered in this paper include parameters such as the horizon H , whose setup requires both domain and task knowledge, and the Thompson-sampling count, which has a similar meaning of t_E and t_Q , but a higher impact especially on the algorithm’s memory requirements.

7 Conclusions

In this work, we introduced QR-MAX, the first model-based RL algorithm for discrete-action NMRDPs that exploits the decoupling between Markovian environment dynamics and non-Markovian rewards. We prove that QR-MAX is PAC-MDP, i.e., all but a polynomial number of interaction steps are ϵ -optimal with high probability, and show on an extended Office grid-world benchmark that it substantially reduces the required interactions compared to both model-free and standard model-based approaches. We also described BUCKET-QR-MAX, which leverages SimHash to handle continuous-state NMRDPs without manual gridding or neural approximators; under mild smoothness and discretization assumptions, it admits a conditional PAC-MDP guarantee (see Appendix). Across our benchmarks, QR-MAX and its variants exhibit higher sample efficiency and scalability, reduced computational cost, and easier hyper-parameter tuning than the evaluated baselines. Future directions include extending the factorization framework to deep RL techniques, handling stochastic RMs [Corazza *et al.*, 2022], learning reward models automatically, and extending the approach to multi-agent scenarios, where reward machines have already been used for cooperative and concurrent multi-agent RL [Neary *et al.*, 2021; Trapasso and Jonsson, 2025].

Acknowledgments

This research is supported by the PNRR MUR project PE0000013- FAIR.

References

- [Azar *et al.*, 2017] Mohammad Gheshlaghi Azar, Ian Osband, and Rémi Munos. Minimax regret bounds for reinforcement learning. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 263–272. PMLR, 06–11 Aug 2017.
- [Bacchus *et al.*, 1996] Fahiem Bacchus, Craig Boutilier, and Adam J. Grove. Rewarding behaviors. In *Proc. AAAI*, pages 1160–1167, 1996.
- [Bernstein and Shimkin, 2008] Andrey Bernstein and Nahum Shimkin. Adaptive aggregation for reinforcement learning with efficient exploration: Deterministic domains. In *COLT*, pages 323–334. Citeseer, 2008.
- [Bourel *et al.*, 2023] Hippolyte Bourel, Anders Jonsson, Odalric-Ambrym Maillard, and Mohammad Sadegh Talebi. Exploration in Reward Machines with Low Regret. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, page 4114–4146, 2023.
- [Bourel *et al.*, 2024] Hippolyte Bourel, Anders Jonsson, Odalric-Ambrym Maillard, Chenxiao Ma, and Mohammad Sadegh Talebi. Provably efficient exploration in reward machines with low regret. *CoRR*, abs/2412.19194, 2024.
- [Brafman and Tennenholtz, 2003] Ronen I. Brafman and Moshe Tennenholtz. R-max - a general polynomial time algorithm for near-optimal reinforcement learning. *J. Mach. Learn. Res.*, 3:213–231, 2003.
- [Brafman *et al.*, 2018] Ronen Brafman, Giuseppe De Giacomo, and Fabio Patrizi. LTLf/LDLf non-Markovian rewards. In *Proc. AAAI*, pages 1771–1778, 2018.
- [Charikar, 2002] Moses S. Charikar. Similarity estimation techniques from rounding algorithms. In *Proceedings of the Thirty-Fourth Annual ACM Symposium on Theory of Computing*, STOC ’02, page 380–388, New York, NY, USA, 2002. Association for Computing Machinery.
- [Cipollone *et al.*, 2025] Roberto Cipollone, Marco Favorito, Flavio Maiorana, Giuseppe De Giacomo, Luca Iocchi, and Fabio Patrizi. Exploiting robot abstractions in episodic RL via reward shaping and heuristics. *Robotics and Autonomous Systems*, 193:105116, November 2025.
- [Corazza *et al.*, 2022] Jan Corazza, Ivan Gavran, and Daniel Neider. Reinforcement learning with stochastic reward machines. In *Proc. AAAI*, pages 6429–6436, 2022.
- [De Giacomo *et al.*, 2020] Giuseppe De Giacomo, Luca Iocchi, Marco Favorito, and Fabio Patrizi. Restraining bolts for reinforcement learning agents. In *Proc. ICAPS*, volume 34, pages 13659–13662, 2020.
- [Gaon and Brafman, 2020] Maor Gaon and Ronen I. Brafman. Reinforcement learning with non-Markovian rewards. In *Proc. AAAI*, pages 3980–3987, 2020.
- [Guestrin *et al.*, 2003] C. Guestrin, D. Koller, R. Parr, and S. Venkataraman. Efficient solution algorithms for factored mdps. *Journal of Artificial Intelligence Research*, 19:399–468, October 2003.
- [Gupta *et al.*, 2021] Gaurav Gupta, Chenzhong Yin, Jyotirmoy V. Deshmukh, and Paul Bogdan. Non-Markovian reinforcement learning using fractional dynamics. In *Proc. CDC*, pages 1542–1547, 2021.
- [Kearns and Koller, 1999] Michael Kearns and Daphne Koller. Efficient reinforcement learning in factored mdps. In *Proceedings of the 16th International Joint Conference on Artificial Intelligence - Volume 2, IJCAI’99*, page 740–747, San Francisco, CA, USA, 1999. Morgan Kaufmann Publishers Inc.
- [Lin and Zhang, 2025] Xiaofeng Lin and Xuezhou Zhang. Efficient reinforcement learning in probabilistic reward machines. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(18):18728–18736, Apr. 2025.
- [Neary *et al.*, 2021] Cyrus Neary, Zhe Xu, Bo Wu, and Ufuk Topcu. Reward machines for cooperative multi-agent reinforcement learning. In *AAMAS ’21: 20th International Conference on Autonomous Agents and Multiagent Systems, Virtual Event, United Kingdom, May 3–7, 2021*, pages 934–942. ACM, 2021.
- [Osband and Roy, 2014] Ian Osband and Benjamin Van Roy. Near-optimal reinforcement learning in factored mdps, 2014.
- [Osband *et al.*, 2013] Ian Osband, Daniel Russo, and Benjamin Van Roy. (more) efficient reinforcement learning via posterior sampling, 2013.
- [Shao and Kwiatkowska, 2023] Daqian Shao and Marta Kwiatkowska. Sample efficient model-free reinforcement learning from LTL specifications with optimality guarantees. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI 2023, 19th-25th August 2023, Macao, SAR, China*, pages 4180–4189. ijcai.org, 2023.
- [Strehl *et al.*, 2009] Alexander L. Strehl, Lihong Li, and Michael L. Littman. Reinforcement learning in finite mdps: PAC analysis. *J. Mach. Learn. Res.*, 10:2413–2444, 2009.
- [Strehl, 2007] Alexander L. Strehl. Model-based reinforcement learning in factored-state mdps. In *Proceedings of the 2007 IEEE International Symposium on Approximate Dynamic Programming and Reinforcement Learning*, pages 103–110. IEEE, 2007.
- [Strens, 2000] Malcolm J. A. Strens. A bayesian framework for reinforcement learning. In *Proceedings of the Seventeenth International Conference on Machine Learning*, ICML ’00, page 943–950, San Francisco, CA, USA, 2000. Morgan Kaufmann Publishers Inc.
- [Sutton and Barto, 2018] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [Thiébaux *et al.*, 2006] Sylvie Thiébaux, Charles Gretton, John K. Slaney, David Price, and Frodoald Kabanza. Decision-theoretic planning with non-markovian rewards. *J. Artif. Intell. Res.*, 25:17–74, 2006.

- [Tiapkin *et al.*, 2022] Daniil Tiapkin, Denis Belomestny, Daniele Calandriello, Eric Moulines, Remi Munos, Alexey Naumov, Mark Rowland, Michal Valko, and Pierre Menard. Optimistic posterior sampling for reinforcement learning with few samples and tight guarantees. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022.
- [Toro Icarte *et al.*, 2022] Rodrigo Toro Icarte, Toryn Q. Klassen, Richard Anthony Valenzano, and Sheila A. McIlraith. Reward machines: Exploiting reward function structure in reinforcement learning. *J. Artif. Intell. Res.*, 73:173–208, 2022.
- [Trapasso and Jonsson, 2025] Alessandro Trapasso and Anders Jonsson. Concurrent multiagent reinforcement learning with reward machines. In *ECAI 2025 - 28th European Conference on Artificial Intelligence*, volume 413 of *Frontiers in Artificial Intelligence and Applications*, pages 3735–3742. IOS Press, 2025.
- [Watkins and Dayan, 1992] Christopher Watkins and Peter Dayan. Q-learning. *Mach. Learn.*, 8(3):279–292, 1992.

A Detailed Proofs for QR-MAX

We give the full proofs of Lemmas A.1–A.2 and Theorem A.3 that were stated without proof in Section 4.2 of the main paper.

A.1 Notation and Preliminaries

Let $N_E = |S| |A|$ and $N_Q = |S| |Q|$. For every environment state-action pair (s, a) and next state s' , define:

$$\begin{aligned}\hat{P}_E(s' | s, a) &= \frac{\tau_E(s, a, s')}{n_E(s, a)}, & \hat{R}_E(s, a, s') &= \frac{\rho_E(s, a, s')}{n_E(s, a)}, \\ \hat{P}_Q(q' | q, s') &= \frac{\tau_Q(q, s', q')}{n_Q(q, s')}, & \hat{R}_Q(q, s', q') &= \frac{\rho_Q(q, s', q')}{n_Q(q, s')}.\end{aligned}$$

All other notation follows Algorithm 1 in the main text.

A.2 Proof of Lemma A.1 (Accurate Estimates)

$\beta = \epsilon(1 - \gamma)/4$ and define the thresholds m_E and m_Q accordingly.

Lemma A.1 (Factored Accuracy). *With probability at least $1 - \delta$, whenever QR-MAX marks a pair known (i.e., (s, a) after m_E visits or (q, s') after m_Q visits), the following simultaneously hold for all (s, a) and (q, s') :*

$$\begin{aligned}|\hat{R}_E(s, a) - \bar{R}_E(s, a)| &\leq \beta, \\ \|\hat{P}_E(\cdot | s, a) - P_E(\cdot | s, a)\|_1 &\leq \frac{\beta}{R_{\max}}, \\ |\hat{R}_Q(q, s') - \bar{R}_Q(q, s')| &\leq \beta, \\ \|\hat{P}_Q(\cdot | q, s') - P_Q(\cdot | q, s')\|_1 &\leq \frac{\beta}{R_{\max}}.\end{aligned}\tag{A.1}$$

Proof. For reward estimation in $[0, R_{\max}]$, Hoeffding’s inequality implies that for m_E i.i.d. samples:

$$\Pr\left[\left|\frac{1}{m_E} \sum_i r_i - \mathbb{E}[r]\right| > \beta\right] \leq 2 \exp(-2m_E \beta^2 / R_{\max}^2).$$

Setting $m_E = \lceil 8R_{\max}^2 \beta^{-2} \ln(2N_E/\delta) \rceil$ bounds this by $\delta/(2N_E)$. A union bound over all N_E pairs ensures all reward estimates are within β with probability at least $1 - \delta/2$.

For transition estimation in ℓ_1 , Weissman’s inequality states that for a distribution over $|S|$ outcomes and m_E samples:

$$\Pr[\|\hat{P}_E - P_E\|_1 > \epsilon_P] \leq (2^{|S|} - 2) \exp(-m_E \epsilon_P^2 / 2).$$

Choosing $\epsilon_P = \beta/R_{\max}$ and requiring $(2^{|S|}) \exp(-m_E \beta^2 / (2R_{\max}^2)) \leq \delta/(2N_E)$ yields a similar threshold on m_E . The same argument applies to the (q, s') counts with m_Q, N_Q . A final union bound gives overall failure probability at most δ . $\square$

A.3 Proof of Lemma A.2 (Near-Optimal Optimistic Policy)

Lemma A.2 (Near-Optimality in the Known Model). *Conditioned on Lemma A.1, the greedy policy computed in the optimistic model is ϵ -optimal whenever no unknown component is reached within the effective horizon H .*

Proof. Condition on the event of Lemma A.1. Consider an execution of the greedy policy in which no unknown environment or automaton component is reached within the effective horizon H . Along this event, all Bellman backups involve only known components. Hence the empirical factorised model and the true model differ only by the reward and transition errors controlled in Lemma A.1.

For any value function V with $\|V\|_\infty \leq V_{\max}$, the one-step simulation bound gives

$$\|\hat{T}V - TV\|_\infty \leq \xi_R + \gamma V_{\max}(\xi_E + \xi_Q),$$

where ξ_R, ξ_E, ξ_Q are the reward, environment-transition, and automaton-transition errors. By the choice of the thresholds and of the value-iteration accuracy η_{VI} , the accumulated discounted error over the effective horizon is at most ϵ . Therefore, conditional on not reaching an unknown component within H , the greedy policy computed in the optimistic known model is ϵ -optimal in the true model. $\square$

A.4 Proof of Theorem 3

Theorem A.3 (PAC-MDP Bound for QR-MAX). *Let*

$$U = |S| |A| m_E + |S| |Q| m_Q.$$

With the thresholds chosen as in Lemma A.1, QR-MAX is PAC-MDP. With probability at least $1 - \delta$, the number of interaction steps in which its greedy policy is not ϵ -optimal is bounded by

$$O\left(\frac{HU}{p} \log \frac{U}{\delta}\right),$$

where

$$H = \left\lceil \frac{1}{1 - \gamma} \ln \frac{2R_{\max}}{\epsilon(1 - \gamma)} \right\rceil, \quad p = \frac{\epsilon(1 - \gamma)}{2R_{\max}}.$$

Proof. Condition on the concentration event of Lemma A.1. If the greedy policy in the optimistic model is not ϵ -optimal, then Lemma A.2 implies that, within the effective horizon H , the policy must reach an unknown component with probability at least p . Each such escape is charged to the first unknown component reached.

Environment components can be charged at most $|S| |A| m_E$ times and automaton components at most $|S| |Q| m_Q$ times. Hence the total number of escapes is at most

$$U = |S| |A| m_E + |S| |Q| m_Q.$$

The standard R-Max escape-event argument then gives

$$O\left(\frac{HU}{p} \log \frac{U}{\delta}\right)$$

non- ϵ -optimal interaction steps with probability at least $1 - \delta$. Value iteration affects computation time but does not consume environment interactions. $\square$

B Bucket-QR-Max: Pseudocode and Conditional PAC-MDP Bound

We now give the full pseudocode for BUCKET-QR-MAX and state a PAC-MDP guarantee in the continuous-state setting, under a standard bounded-discretisation assumption. The

Algorithm 2. BUCKET-QR-MAX

Input: Hash map $h : \mathbb{R}^d \rightarrow \mathcal{B}$, action set A , automaton states Q , environment E , discount factor γ , maximum reward $R_{\max}$, thresholds $t_{E_T}, t_{E_R}, t_{Q_T}, t_{Q_R}$, VI horizon T , number of steps N_{steps} .

Output: Optimistic action-value table Q_B .

$V_{\max} \leftarrow R_{\max}/(1 - \gamma)$
 $\mathcal{B}_{\text{disc}} \leftarrow \emptyset$ // Discovered buckets

Initialize all counters lazily to zero: $\tau_E(b, a, b'), n_{E_T}(b, a), \rho_E(b, a), n_{E_R}(b, a), \tau_Q(q, b', q'), n_{Q_T}(q, b'), \rho_Q(q, b'), n_{Q_R}(q, b')$

Function AddBucket(b):

```

    if  $b \notin \mathcal{B}_{\text{disc}}$  then
         $\mathcal{B}_{\text{disc}} \leftarrow \mathcal{B}_{\text{disc}} \cup \{b\}$ 
         $Q_B(b, q, a) \leftarrow V_{\max}$  for all  $q \in Q, a \in A$ 
        Initialize all counters involving  $b$  to zero

```

$s^{\text{cont}}, q \leftarrow E.\text{reset}()$
 $b \leftarrow h(s^{\text{cont}})$
AddBucket(b)

for $t \leftarrow 1$ **to** N_{steps} **do**

```

     $\bar{a} \leftarrow \arg \max_{a \in A} Q_B(b, q, a)$ 
     $s^{\text{cont}'}, q', r_E, r_A, \text{done} \leftarrow E.\text{step}(\bar{a})$ 
     $b' \leftarrow h(s^{\text{cont}'})$ 
    AddBucket( $b'$ )
    doVI  $\leftarrow$  False

    // Environment transition component
    if  $n_{E_T}(b, \bar{a}) < t_{E_T}$  then
         $\tau_E(b, \bar{a}, b') \leftarrow \tau_E(b, \bar{a}, b') + 1$ 
         $n_{E_T}(b, \bar{a}) \leftarrow n_{E_T}(b, \bar{a}) + 1$ 
        if  $n_{E_T}(b, \bar{a}) = t_{E_T}$  then
            doVI  $\leftarrow$  True

    // Environment reward component
    if  $n_{E_R}(b, \bar{a}) < t_{E_R}$  then
         $\rho_E(b, \bar{a}) \leftarrow \rho_E(b, \bar{a}) + r_E$ 
         $n_{E_R}(b, \bar{a}) \leftarrow n_{E_R}(b, \bar{a}) + 1$ 
        if  $n_{E_R}(b, \bar{a}) = t_{E_R}$  then
            doVI  $\leftarrow$  True

    // Automaton transition component
    if  $n_{Q_T}(q, b') < t_{Q_T}$  then
         $\tau_Q(q, b', q') \leftarrow \tau_Q(q, b', q') + 1$ 
         $n_{Q_T}(q, b') \leftarrow n_{Q_T}(q, b') + 1$ 
        if  $n_{Q_T}(q, b') = t_{Q_T}$  then
            doVI  $\leftarrow$  True

    // Non-Markovian reward component
    if  $n_{Q_R}(q, b') < t_{Q_R}$  then
         $\rho_Q(q, b') \leftarrow \rho_Q(q, b') + r_A$ 
         $n_{Q_R}(q, b') \leftarrow n_{Q_R}(q, b') + 1$ 
        if  $n_{Q_R}(q, b') = t_{Q_R}$  then
            doVI  $\leftarrow$  True

    if done then
         $Q_B(b', q', a') \leftarrow 0$  for all  $a' \in A$ 

    if doVI then
         $Q_B \leftarrow$ 
            BucketVI( $Q_B, \tau_E, \rho_E, n_{E_T}, n_{E_R}, \tau_Q, \rho_Q, n_{Q_T}, n_{Q_R}, \gamma, T$ )

    if done then
         $s^{\text{cont}}, q \leftarrow E.\text{reset}()$ 
         $b \leftarrow h(s^{\text{cont}})$ 
        AddBucket( $b$ )
    else
         $s^{\text{cont}}, q, b \leftarrow s^{\text{cont}'}, q', b'$ 

```

return Q_B

proof proceeds by viewing BUCKET-QR-MAX as QR-MAX on the abstract MDP induced by the buckets, and then controlling the approximation error due to discretisation.

The routine BucketVI performs optimistic value iteration on the finite abstraction $\mathcal{B}_{\text{disc}} \times Q$. The empirical estimates used in the backup are

$$\hat{P}_E(b' | b, a) = \frac{\tau_E(b, a, b')}{n_{E_T}(b, a)}, \quad \hat{r}_E(b, a) = \frac{\rho_E(b, a)}{n_{E_R}(b, a)},$$

and

$$\hat{P}_Q(q' | q, b') = \frac{\tau_Q(q, b', q')}{n_{Q_T}(q, b')}, \quad \hat{r}_Q(q, b') = \frac{\rho_Q(q, b')}{n_{Q_R}(q, b')}.$$

A triple (b, q, a) is treated as known only when $n_{E_T}(b, a) \geq t_{E_T}$, $n_{E_R}(b, a) \geq t_{E_R}$, and, for every observed successor bucket b' with $\tau_E(b, a, b') > 0$, $n_{Q_T}(q, b') \geq t_{Q_T}$ and $n_{Q_R}(q, b') \geq t_{Q_R}$.

For compactness, let

$$V_k(b, q) := \max_{a \in A} Q_k(b, q, a),$$

and define

$$G_k(q, b') := \hat{r}_Q(q, b') + \gamma \sum_{q' \in Q} \hat{P}_Q(q' | q, b') V_k(b', q').$$

For known triples, BucketVI applies the factorised Bellman backup

$$Q_{k+1}(b, q, a) = \hat{r}_E(b, a) + \sum_{b' \in \mathcal{B}_{\text{disc}}} \hat{P}_E(b' | b, a) G_k(q, b'). \quad (\text{B.1})$$

Unknown triples retain their optimistic value $V_{\max} = R_{\max}/(1 - \gamma)$.

B.1 Bounded-Discretisation Assumption

We formalise the regularity condition under which the bucket abstraction approximates the underlying continuous MDP closely enough.

Assumption B.1 (Bounded discretisation error). *Let M be the true MDP with continuous state space $S \subseteq \mathbb{R}^d$ and let $h : S \rightarrow \mathcal{B}$ be the SimHash-based discretisation used by BUCKET-QRMAX, inducing a partition $\mathcal{B} = \{b_1, \dots, b_{|\mathcal{B}|}\}$ of S . We assume that there exists a diameter parameter $\phi > 0$ such that, for every bucket $b \in \mathcal{B}$ and any pair of states $s, s' \in S$ with $h(s) = h(s') = b$, we have*

$$\|s - s'\| \leq \phi.$$

Moreover, we assume that the reward and transition dynamics are Lipschitz-continuous in the state, i.e., there exists $L > 0$ such that for all $s, s' \in S$ and all $a \in A$,

$$|R_E(x, a, z) - R_E(y, a, z)| \leq L \|x - y\| \quad \forall x, y, z \in S$$

and the transition kernels satisfy a corresponding Lipschitz condition (e.g., in total-variation or Wasserstein distance) with constant L .

Assumption B.1 captures the intuition that each bucket groups together nearby states and that the MDP is sufficiently smooth in the state. In particular, if ϕ is small, then the error incurred by replacing s with its bucket $b = h(s)$ is also small.

B.2 Conditional PAC-MDP Bound

We now show that, under Assumption B.1, the discretisation error can be absorbed into the standard PAC-MDP analysis, yielding a conditional PAC guarantee for BUCKET-QRMAX.

Theorem B.2 (Conditional PAC-MDP Bound for BUCKET-QRMAX). *Assume B.1 holds. Let $\varepsilon > 0$ and $\delta \in (0, 1)$, and set*

$$\beta = \frac{\varepsilon(1 - \gamma)}{8}.$$

Choose thresholds

$$m_{E_T}, m_{E_R} = \left\lceil \frac{8R_{\max}^2}{\beta^2} \ln \frac{2|\mathcal{B}||\mathcal{A}|}{\delta} \right\rceil,$$

$$m_{Q_T}, m_{Q_R} = \left\lceil \frac{8R_{\max}^2}{\beta^2} \ln \frac{2|\mathcal{B}||\mathcal{Q}|}{\delta} \right\rceil.$$

Let

$$U_B = |\mathcal{B}||\mathcal{A}|(m_{E_T} + m_{E_R}) + |\mathcal{B}||\mathcal{Q}|(m_{Q_T} + m_{Q_R}).$$

Moreover, let

$$H = \left\lceil \frac{1}{1 - \gamma} \ln \frac{2R_{\max}}{\varepsilon(1 - \gamma)} \right\rceil, \quad p = \frac{\varepsilon(1 - \gamma)}{2R_{\max}}.$$

Suppose that the bucket diameter satisfies $\phi \leq c\beta/L$ for a suitable universal constant $c > 0$. Then, with probability at least $1 - \delta$, BUCKET-QRMAX executes at most

$$O\left(\frac{HU_B}{p} \log \frac{U_B}{\delta}\right)$$

interactions in which its policy is not ε -optimal in the true continuous MDP M , up to the discretisation error controlled by Assumption B.1.

1. Discretisation Error and Abstract MDP. Let M be the true continuous-state MDP and let $\bar{M}$ be the abstract MDP whose states are the bucket-automaton pairs $(b, q) \in \mathcal{B} \times \mathcal{Q}$, with transitions and rewards given by the bucket-averaged quantities. Under Assumption B.1, any two states s, s' mapped to the same bucket b satisfy $\|s - s'\| \leq \phi$. Lipschitz continuity then implies that the reward and transition probabilities of M at s and s' differ by at most $O(L\phi)$.

By a standard simulation-lemma argument, there exists a universal constant $c > 0$ such that, if $\phi \leq c\beta/L$, then $\bar{M}$ is a β -approximation of M : for any policy π and any state s with bucket $b = h(s)$,

$$|V_M^\pi(s) - V_M^\pi(b, q)| \leq \frac{\beta}{2}.$$

2. Estimation Error in the Bucket MDP. Within the abstract MDP $\bar{M}$ on (b, q) , BUCKET-QRMAX behaves as QR-MAX on a discrete MDP with state space $\mathcal{B} \times \mathcal{Q}$. The thresholds $m_{E_T}, m_{E_R}, m_{Q_T}, m_{Q_R}$ are chosen so that, once a triple (b, a, q) is marked known, the empirical estimates are β -accurate with high probability. More precisely, a concentration argument and a union bound over all (b, a) and (q, b') yield

$$\|\hat{P} - P\|_1 \leq \frac{\beta}{R_{\max}}, \quad |\hat{R} - R| \leq \beta$$

for all known components, with failure probability at most δ .

Conditioned on this event, applying Theorem A.3 to the abstract bucket MDP gives the same optimistic escape bound with

$$U_B = |\mathcal{B}||\mathcal{A}|(m_{E_T} + m_{E_R}) + |\mathcal{B}||\mathcal{Q}|(m_{Q_T} + m_{Q_R}).$$

Thus the greedy policy is β -optimal in $\bar{M}$, except for

$$O\left(\frac{HU_B}{p} \log \frac{U_B}{\delta}\right)$$

interaction steps.

3. Combining the Two Sources of Error. For any state s with bucket $b = h(s)$, decompose the value gap as

$$\begin{aligned} |V_M^{\pi_t}(s) - V_M^*(s)| &\leq |V_M^{\pi_t}(s) - V_M^{\pi_t}(b, q)| \\ &\quad + |V_M^{\pi_t}(b, q) - V_M^*(b, q)| \\ &\quad + |V_M^*(b, q) - V_M^*(s)|. \end{aligned}$$

The two discretisation terms are controlled by Step 1, while the estimation term is controlled by Step 2 on all non-escape steps. Choosing the constants in the thresholds so that these three contributions sum to at most ε yields the stated guarantee. Therefore the same optimistic escape-event argument gives

$$O\left(\frac{HU_B}{p} \log \frac{U_B}{\delta}\right)$$

non- ε -optimal interactions.

Remark B.3 (On the role of SimHash). *Assumption B.1 is a property of the partition $\mathcal{B}$ induced by the discretisation. In our implementation the partition is generated by a SimHash-based locality-sensitive hash. SimHash does not deterministically guarantee a given diameter ϕ , but increasing the number of hash functions and projections reduces the probability that distant states collide in the same bucket. Thus, BUCKET-QRMAX can be viewed as satisfying a PAC-type property with probability at least $1 - (\delta + \delta_{\text{LSH}})$, where δ_{LSH} denotes the probability that the induced partition violates Assumption B.1. In the main text we focus on the PAC bound for the induced bucket MDP, and leave a detailed analysis of δ_{LSH} to future work.*

C Additional Experimental Results

The experiments were performed on a Linux x86-64 workstation with a single-socket AMD Ryzen 9 7950X processor (Zen 4, 16 cores/32 threads, boost clock ~ 5.9 GHz). The system is equipped with 128 GB of DDR5 RAM (≈ 124 GiB available at idle) and an 8 GB swap partition. Hardware acceleration is provided by two NVIDIA GeForce RTX 4090 (AD102) GPUs. This configuration underlies all the results presented in our work.

C.1 Office gridworld Benchmark

The benchmark used for experimental comparison in this paper takes inspiration from and extends the Office gridworld environment [Toro Icarte *et al.*, 2022] already used to evaluate solutions for NMRDPs. We extend the original environment with the following features: (1) stochastic outcomes of

actions: the nominal effect of an action is achieved with probability h , and two adjacent actions with probability $(1 - h)/2$; (2) three different sizes of the map 9×12 , 12×12 , and 15×15 (the latter is shown in Figure 10); additional locations to be used as goals; the definition of 5 different tasks of increasing complexity. The benchmark can be used to test algorithms in 15 environments (5 tasks in 3 maps) with increasing complexity.

In the first task, the agent must deliver coffee to the office. In the second task, the goal is to deliver mail to the office. The third task requires the agent to deliver both coffee and mail to the office in any order. The fourth task requires the agent to visit locations A, B, C, and D in the correct sequence. Finally, in the fifth task, the agent must first visit locations A, B, C, and D, then collect both mail and coffee in any order, and ultimately reach the office (the DFA for the latter task is illustrated in Figure 15).

The agent observation space is given by its own position extended with the state of the DFA, i.e., $(s, q) \in S \times Q$. The action space is formed by the four cardinal actions. The reward signal is 1 for completing the task (i.e., for goal states in $\mathcal{A}$) and 0 otherwise. Walls make the agent remain in the same cell, while hitting decorations terminates the episode with reward -100 .

C.2 Environment map0.

We build on the grid-world map0 introduced by [Tiapkin *et al.*, 2022], a 10×10 grid. For UCBVI, PSRL, and OPSRL the planning horizon is fixed at $H = 50$; the two Bayesian methods use eight Thompson samples per episode, following the authors’ default.

Non-Markovian extension. The original one-shot objective is replaced by a four-stage mission that forces the agent to remember past events. The episode begins in the top-left cell; the agent must first travel to the bottom-right cell to pick up a letter, then return to the start cell to collect a coffee, and finally deliver both items to the office located in the bottom-left corner. A reward of 1 is granted only when this entire sequence is completed; every other transition yields zero reward.

C.3 Comparison algorithms

Our evaluation includes both optimistic planners and posterior-sampling methods defined over the joint space $S \times Q$. We adopt the three variants of UCBVI proposed by [Azar *et al.*, 2017]: UCBVI-sB uses the simplified Bernstein bonus, UCBVI-H the Hoeffding bonus, and UCBVI-B the full Bernstein bonus; in each case planning is performed with tabular value iteration and the same optimism constant c . To represent the Bayesian family we run PSRL [Osband *et al.*, 2013], which samples one MDP from a Dirichlet-Gaussian posterior at the start of every episode, and OPSRL [Tiapkin *et al.*, 2022], which augments each posterior sample with an explicit exploration bonus. As a classical optimistic baseline we include R-MAX [Brafman and Tennenholtz, 2003]. For the non-Markovian setting we also test the model-free algorithm QRM [Toro Icarte *et al.*, 2022], which learns a Q-function directly over $S \times Q$.

The algorithm introduced in this paper, QR-MAX, exploits the factorisation of the environment and automaton dynamics,

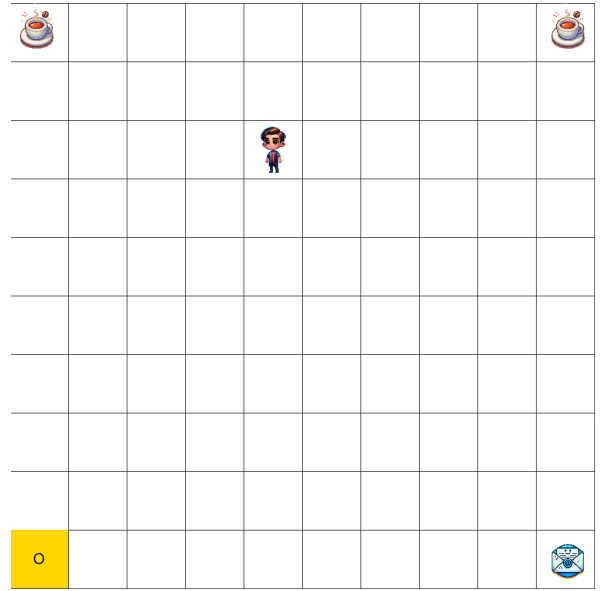


Figure 4: Map0 - task 0 (10x10)

while R-MAXRM and QR-MAXRM extend R-MAX and QR-MAX, respectively, by propagating knowledge to unseen states as in QRM.

The algorithms QL, R-MAX, QR-MAX, R-MAXRM and QR-MAXRM use the discount factor $\gamma = 0.9$. All the variants of UCBVI and PSRL/OPSRL use the discount factor $\gamma = 1$. For R-MAX, QR-MAX, R-MAXRM and QR-MAXRM, a transition is considered *known* after 39 visits. QRM and Q-LEARNING employ an exploration rate $\epsilon = 0.1$ and a learning rate $\alpha = 0.1$. Following the original papers, PSRL and OPSRL draw eight Thompson samples per episode, and the planning horizon is set to $H = 50$ for map0 and $H = 150$ for map1.

Horizon and reward scheme. All finite-horizon exploration methods—UCBVI (every variant), PSRL and OPSRL—require a hand-chosen horizon H . To give these baselines favourable yet realistic conditions we set H to the *average optimal-path length* computed by Value Iteration, adding a small safety margin for stochastic slips: $H = 50$ in MAP0.EXP0, $H = 150$ in MAP1.EXP5 and MAP4.EXP6, and $H = 350$ in MAP2.EXP5 (Table 4). Choosing H too low truncates the policy, while over-estimating it inflates exploration bonuses and slows learning at a rate of $\tilde{O}(H^{2-4})$; the calibration therefore embodies non-trivial prior knowledge that horizon-free algorithms do not need.

A uniform, deliberately sparse reward is used throughout: the agent receives $+1$ only when the Reward-Machine task is fully completed and -100 upon hitting a decoration; all other transitions yield zero. The strong terminal penalty restarts the episode immediately, ensuring that no method—irrespective of its dependence on H —wastes steps in an absorbing state. This design isolates exploration efficiency as the sole differentiator between algorithms and prevents any hidden advantage from reward shaping.

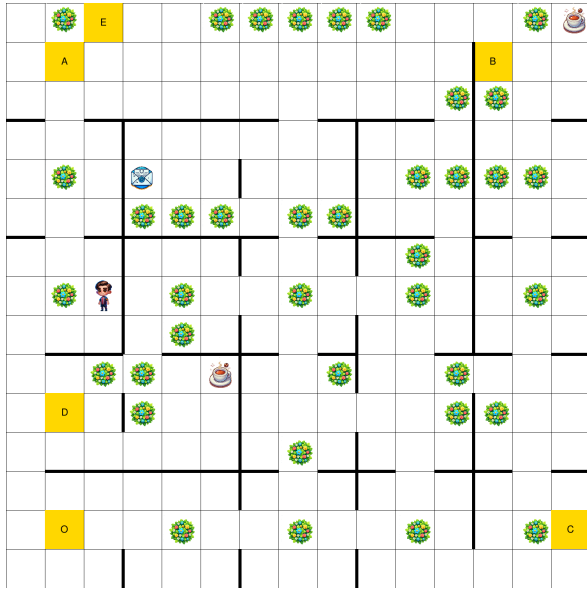


Figure 5: Map4 - task 6 (15x15) - Office World

Algorithm	Type	Dynamics factorized	DFA known
Q-LEARNING	model-free	✗	✗
R-MAX	model-based	✗	✗
QR-MAX	model-based	✓	✗
QRM	model-free	✗	✓
R-MAXRM	model-based	✗	✓
QR-MAXRM	model-based	✓	✓

Table 2: Compared algorithms.

C.4 Overall results on 15 configurations

Figure 6 and Table 4 reports, for each *map-task* instance, the *training steps* (log scale, mean $\pm$ std. dev.) required by each algorithm to learn a policy that is optimal in the corresponding deterministic MDP. Results are averaged over thirty runs with 30 different random seeds, and at regular intervals the current policy is evaluated over 100 deterministic episodes; training stops as soon as its mean return is statistically indistinguishable from the optimal one ($p > 0.1$, two-sided Welch’s t -test).

- **Model-based vs. model-free:** both R-MAX and QR-MAX consistently require far fewer steps than their model-free counterparts (Q-LEARNING, QRM), regardless of DFA access.
- **Benefit of knowing the DFA:** supplying the reward-machine structure (suffix *RM*) yields an additional 10–50 $\times$ reduction in sample complexity (R-MAXRM, QR-MAXRM vs. R-MAX, QR-MAX).
- **Impact of the factorisation:** even with DFA knowledge, QR-MAXRM is about 1.5–2 $\times$ faster than R-MAXRM, and without DFA QR-MAX is roughly an order of magnitude faster than R-MAX, showing that sharing the learned environment model across automaton states remains advantageous.
- **Overall winner:** QR-MAXRM achieves the lowest step count on every instance; even without the DFA, QR-MAX outperforms QRM, confirming that the factorisa-

tion compensates for the lack of reward-machine knowledge.

In summary, factorising environment and reward dynamics delivers substantial and orthogonal gains over both classic R-MAX and reward-machine baselines, yielding the most sample-efficient algorithm across all Office-World configurations (Figure 7).

C.5 Detailed results on 5 configurations

In this section, we analyze in detail the behavior of the different algorithms on 5 relevant configurations of the Office gridworld environment. In particular, we used three distinct maps and five different Reward Machines, each progressively more complex. Map 1 in figure 8 is a 12x9 grid, the same used by Icarte et al. (2022). Map 2 in figure 9 is a 12x12 grid, featuring more rooms and decorations. Map 3 in figure 10 is the most complex, a 15x15 maze-like grid with goals arranged to challenge the agent. All environments share the same discrete action set—*up*, *down*, *left*, *right*—and a stochastic transition noise of 0.2. In *map0_exp0* (a 10 $\times$ 10 empty grid), each action has a 20% chance of being replaced by any other action. In *map1_exp5* and *map4_exp6* the noise deflects the agent towards a perpendicular action, mirroring the *slip* mechanism of FROZEN LAKE.

The agent’s objective is to reach the goals in a specific order without stepping on any decorations, which would terminate the episode.

The five Reward Machines correspond to five experiments with increasing difficulty:

- **Task 1:** Deliver coffee to the office without breaking any decoration (figure 11).
- **Task 2:** Deliver mail to the office without breaking any decoration (figure 12).
- **Task 3:** Deliver both coffee and mail to the office, in any order, without breaking any decoration (figure 13).
- **Task 4:** Patrol locations A, B, C, and D in the correct order, without breaking any decoration (figure 14).
- **Task 5:** Patrol locations A, B, C, D, then collect email and coffee in any order, and finally go to the office (figure 15).

In addition, we consider a sixth task (Task 6), defined only on the largest map (Map4), where the environment is a maze-like space featuring two coffee machines: a *good* one (reward 1000) and a *regular* one (reward 1); see Figure 16.

Table 3 reports the mean and standard deviation of the number of training steps required by each algorithm to reach 100% success, averaged over the five experiments (exp1–exp5) on each map. As the table shows, the model-free baseline Q-Learning incurs extremely high sample complexity (on the order of 5×10^5 to 1.8×10^6 steps) with very large variance, particularly on the larger maps. Introducing a model (R-MAX) reduces the mean by a factor of roughly 3–6, while QR-MAX further cuts it by another factor of ≈ 4 , achieving means of 3.1×10^4 , 6.9×10^4 , and 1.16×10^5 on Maps 1–3 respectively. Equipping the solvers with the reward-machine structure (the “RM” variants) yields additional speedups: R-MAXRM requires on average only 2.7×10^4 , 6.0×10^4 , and 9.8×10^4 steps, while QR-MAXRM plunges to 7.3×10^3 , 1.6×10^4 ,

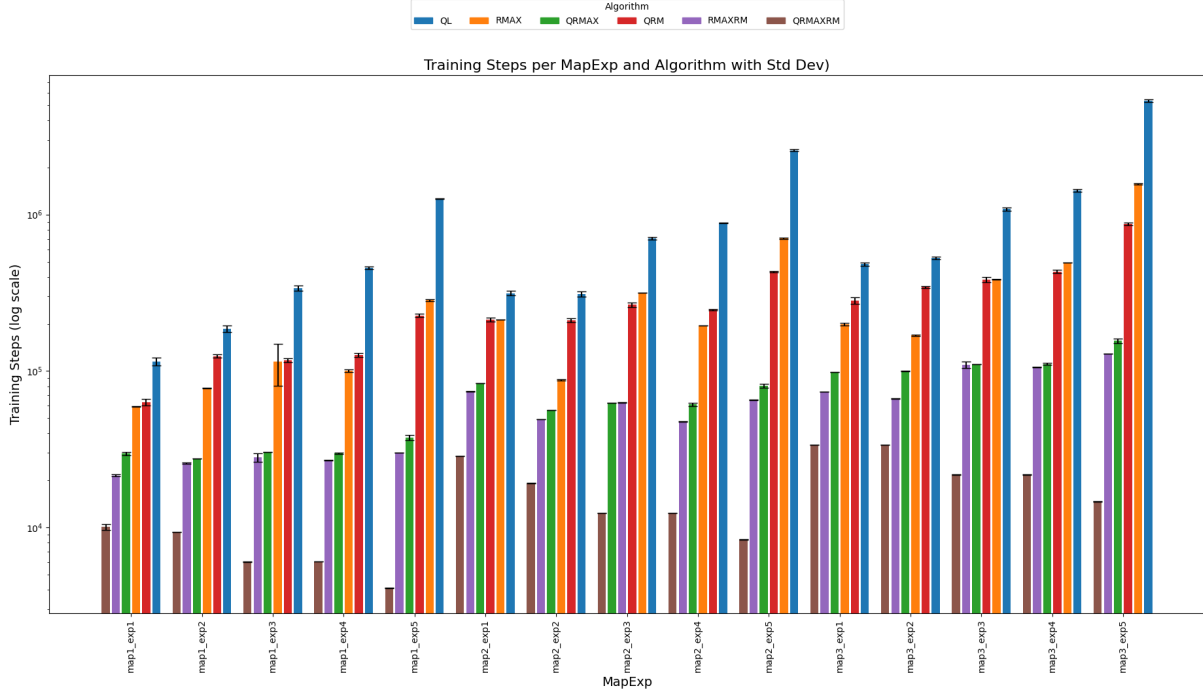


Figure 6: Scalability with respect to task complexity. Training steps needed to achieve the goal in 15 different tasks.

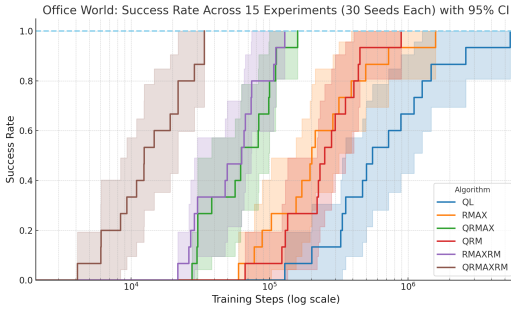


Figure 7: Office World - 15 tasks with 95 % confidence intervals. and 2.5×10^4 steps on Maps 1–3, with relatively low standard deviations (see Table 3).

Table 4 presents the full breakdown by map and experiment, together with the optimal path length and the success rate of value iteration (VI). For example, on Map1 exp1 the optimal makespan is 21.08 steps (VI success 66.76%), and QR-MAXRM converges in 10,964 steps—an order of magnitude fewer than R-MAXRM (21,777) and two orders fewer than Q-Learning (128,973). As task complexity grows (e.g. exp5 on each map), the optimal path length increases substantially (up to 156.46 steps on Map3 exp5) and the VI success rate falls below 10%, reflecting harder coordination challenges. Nevertheless, QR-MAXRM still completes training in 1.47×10^4 steps on Map3 exp5, compared to 1.29×10^5 for R-MAXRM and 5.47×10^6 for Q-Learning.

Taken together, these results demonstrate that factorising environment and reward dynamics, combined with reward-machine knowledge, drives dramatic reductions in sample



Figure 8: Map1 (12x9) - Office World

complexity across all environments and difficulty levels, while the per-experiment metrics in Table 4 highlight the scalability and robustness of QR-MAXRM even as the optimal plans lengthen and the domain becomes more challenging.

Discussion of the results. As summarised in Table 6, the factorised agent QR-Max consistently attains a policy statistically indistinguishable from the value-iteration optimum after roughly one order of magnitude fewer interactions than the strongest optimistic baselines. On the easiest instance (MAP0--EXP0) QR-Max converges in about 1.4×10^4 transitions, whereas the most sample-efficient UCBVI variant requires over 1.4×10^5 ; on the hardest instance (MAP3--EXP5)



Figure 9: Map2 (12x12) - Office World

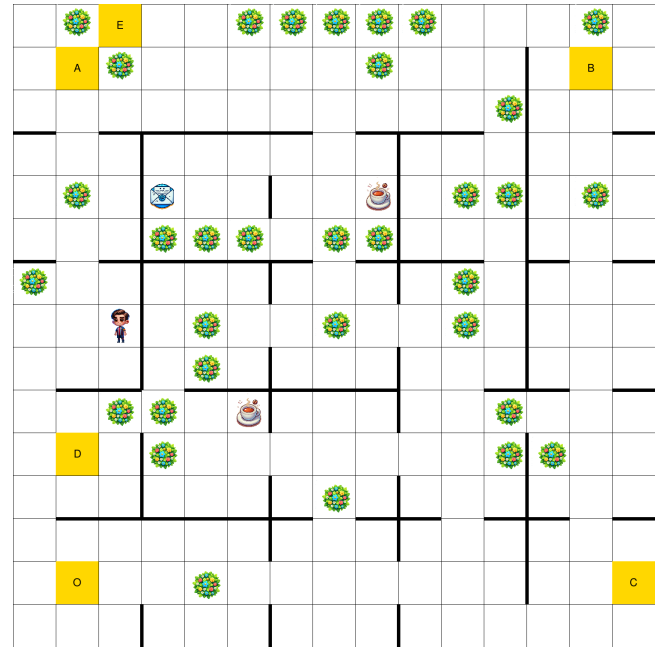


Figure 10: Map3 (15x15) - Office World

Table 3: Mean $\pm$ Std. dev. of training steps to reach 100% success, aggregated over five experiments (exp1–exp5) per map

Algorithm	Map 1	Map 2	Map 3
Q-Learning	484,528 $\pm$ 457,245	977,552 $\pm$ 946,899	1,819,687 $\pm$ 2,080,127
R-MAX	136,622 $\pm$ 91,486	307,805 $\pm$ 246,020	567,680 $\pm$ 582,237
QR-MAX	31,316 $\pm$ 4,073	69,494 $\pm$ 12,854	116,237 $\pm$ 25,001
QRM	137,263 $\pm$ 59,580	282,943 $\pm$ 90,035	479,702 $\pm$ 236,923
R-MAXRM	26,776 $\pm$ 3,199	60,058 $\pm$ 11,165	97,639 $\pm$ 26,300
QR-MAXRM	7,312 $\pm$ 2,774	16,240 $\pm$ 7,990	25,222 $\pm$ 8,385

the respective figures are 1.6×10^5 versus at least 1.4×10^6 .

When the Reward-Machine structure is supplied, QR-MAXRM delivers an even steeper reduction. Across the four largest benchmarks it converges in 3.8×10^3 to 5.8×10^3 interactions, while the model-free QRM requires from half a million to ten million and the non-factorised R-MAXRM more than 1×10^5 . Hence, coupling factorisation with explicit knowledge of the logical task cuts sample complexity by two, and in one case three, orders of magnitude relative to the best published alternatives. These results position QR-MAX as the best when the reward structure is unknown, and establish QR-MAXRM as the most sample-efficient algorithm presently available when the Reward Machine is given.

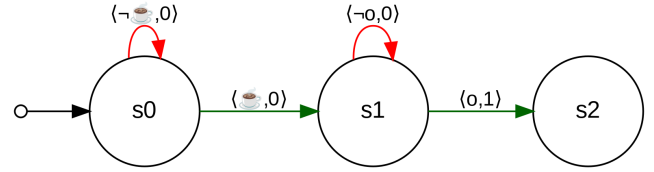


Figure 11: Reward Machine of Experiment 1 - Office World

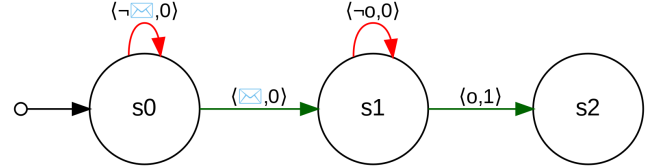


Figure 12: Reward Machine of Experiment 2 - Office World

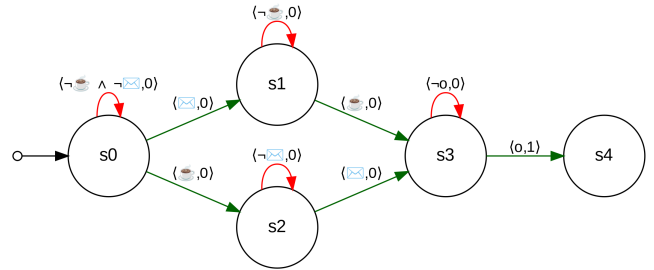


Figure 13: Reward Machine of Experiment 3 - Office World

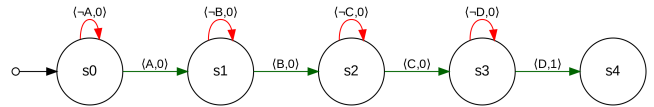


Figure 14: Reward Machine of Experiment 4 - Office World

Table 4: Training steps to reach the optimal policy compared with Value Iteration (VI). The last two columns show the performance of the optimal solutions computed through Value Iteration, highlighting the increasing difficulty of the task.

Map	Exp	Q-Learning	R-MAX	QR-MAX	QRM	R-MAXRM	QR-MAXRM	Average length (VI)	Success rate (VI) (%)
Map1	exp1	128,973	59,691	30,376	66,693	21,777	10,964	21.08	66.76
Map1	exp2	201,835	77,797	27,619	129,804	26,232	9,339	39.31	39.40
Map1	exp3	355,070	155,347	30,325	122,827	28,682	6,087	40.94	38.68
Map1	exp4	469,313	103,211	29,950	135,119	26,953	6,062	42.23	34.37
Map1	exp5	1,267,449	287,066	38,312	231,870	30,238	4,107	77.86	15.47
Map2	exp1	334,894	213,942	83,593	225,491	74,250	28,729	52.46	54.69
Map2	exp2	326,306	88,636	56,445	220,334	49,577	19,214	53.10	37.92
Map2	exp3	723,051	316,672	62,583	280,763	63,066	12,415	63.04	37.25
Map2	exp4	889,842	196,334	61,775	249,912	47,873	12,445	73.80	28.18
Map2	exp5	2,613,669	723,441	83,076	438,213	65,523	8,396	123.39	14.15
Map3	exp1	501,559	201,731	98,663	297,975	73,832	33,815	55.83	33.32
Map3	exp2	553,517	170,926	99,952	353,658	66,949	33,853	63.76	32.57
Map3	exp3	1,105,417	388,562	110,808	406,122	111,349	21,856	68.69	32.42
Map3	exp4	1,466,896	495,878	112,165	449,594	107,115	21,869	81.91	21.27
Map3	exp5	5,471,046	1,581,301	159,597	891,160	128,950	14,717	156.46	6.55

Method	Memory	H -dep.?	Sample complexity*	Key drivers
QR-MAX (ours)	$\tilde{O}(S ^2 A + S Q)$	No	$\tilde{O}(S A + S Q)$	(s, a, s') counts + automaton counts (q, s', q')
UCBVI	$\tilde{O}(H S ^2 A)$	Yes	$\tilde{O}(S A H^3)^\dagger$	Empirical counts and UCB bonuses for every time-step (h, s, a)
OPSLR / PSRL	$\tilde{O}(H S ^2 A)$	Yes	— (high-probability regret only)	Posterior parameters (Dirichlet / Gaussian) for every (h, s, a)
R-MAX [‡]	$\tilde{O}(S ^2 A Q ^2)$	No	$\tilde{O}(S A Q)$	Counts on the full product MDP $S \times Q$
QRM	$\tilde{O}(S A Q)$	No	— (no PAC)	One Q -value table per (s, q, a)

Table 5: Asymptotic memory and (when available) PAC-style sample complexity. $\tilde{O}$ hides poly-log factors; H is the episode horizon, Q the number of reward-machine states. [†]Based on Azar et al. (2017), Thm. 2, combined with the standard conversion from regret to PAC-style bounds (omitting the dependence on ϵ). [‡]Brafman & Tennenholtz (2002).

*Sample-complexity entries are given in terms of $|S|$, $|A|$, $|Q|$ and H , omitting the explicit dependence on ϵ and $(1 - \gamma)$ for clarity.

Reference scenario — 15×15 grid: $|S|=225$, $|A|=4$, $Q=8$, $H=250$.

• QR-MAX memory: $225^2 \times 4 + 225 \times 8 = 0.204$ M entries. • OPSRL/PSRL memory: $250 \times 225^2 \times 4 = 50.6$ M. $\Rightarrow \approx 250 \times$ less memory.
• QR-MAX PAC bound (in $|S|, |Q|$): $225 \times 4 + 225 \times 8 = 2.7$ k. • UCBVI PAC-style bound (in $|S|, H$): $225 \times 4 \times 250^3 \approx 3.5 \times 10^{12}$. $\Rightarrow \approx 10^9 \times$ fewer samples.

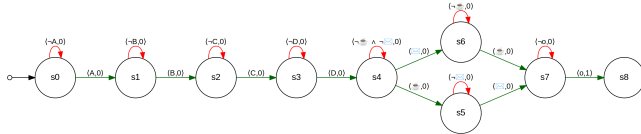


Figure 15: Reward Machine of Experiment 5 - Office World

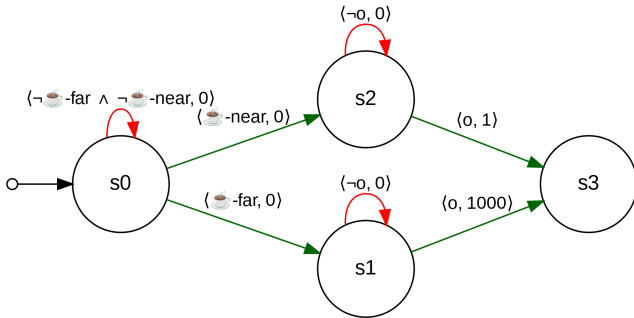


Figure 16: Reward Machine of Experiment 6 - Office World

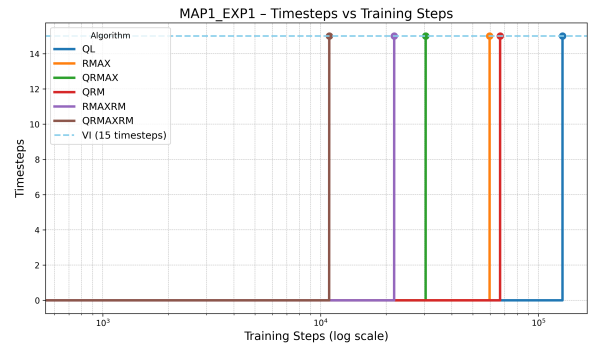


Figure 17: Map1 Exp1 - Office World

Table 6: Average training steps to find a policy that produces results indistinguishable from VI optimal solution (mean over 10 random seeds).

Map	Exp	QR-MAX	QR-MAXRM	R-MAX	QRM	UCBVI-sB	UCBVI-B	UCBVI-H	PSRL/OPSRL [†]
MAP0	EXP0	14 483	4 150	49 759	162 040	142 015	159 092	191 983	88 437 / 128 188
MAP1	EXP5	24 222	3 125	272 080	225 140	250 800	555 977	1 320 070	—
MAP4	EXP6	20 150	5 630	82 000	9 910 340	80 160	90 030	101 120	—
MAP2	EXP5	83 076	3 767	723 441	438 213	1 150 000	1 400 000	1 890 000	—
MAP3	EXP5	159 597	5 806	1 581 301	891 160	1 410 000	1 620 000	2 030 000	—

[†]Eight posterior samples per episode, as in the original papers.

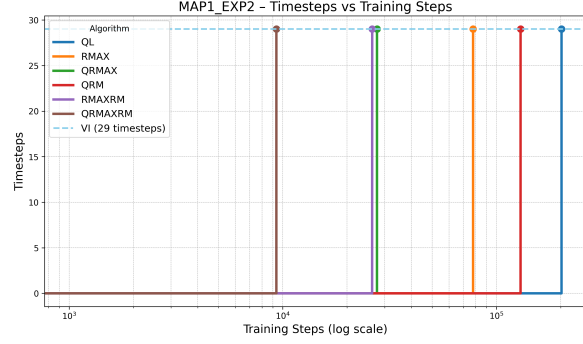


Figure 18: Map1 Exp2 - Office World

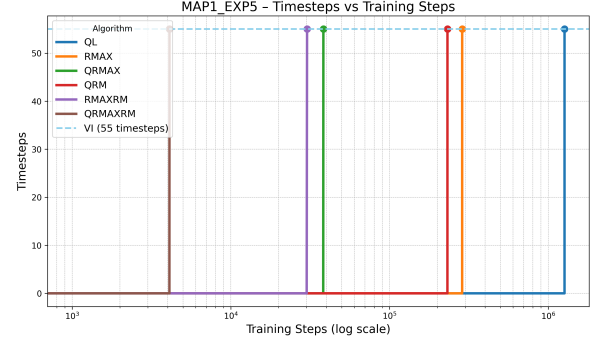


Figure 21: Map1 Exp5 - Office World

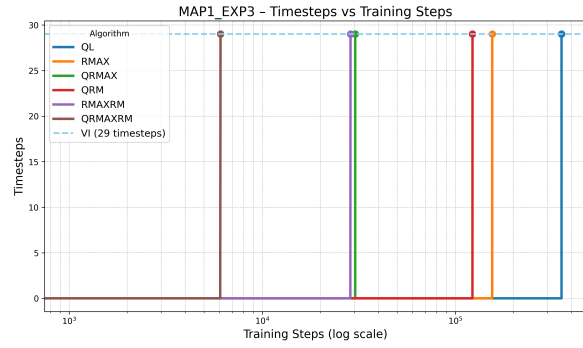


Figure 19: Map1 Exp3 - Office World

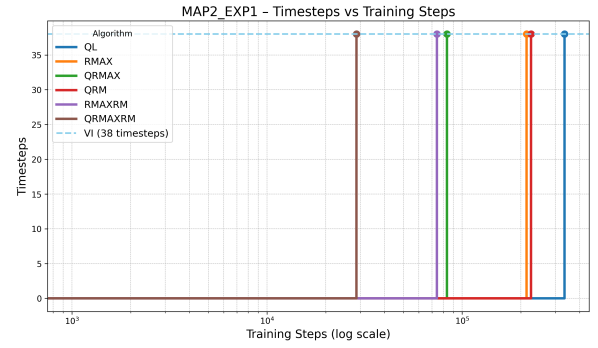


Figure 22: Map2 Exp1 - Office World

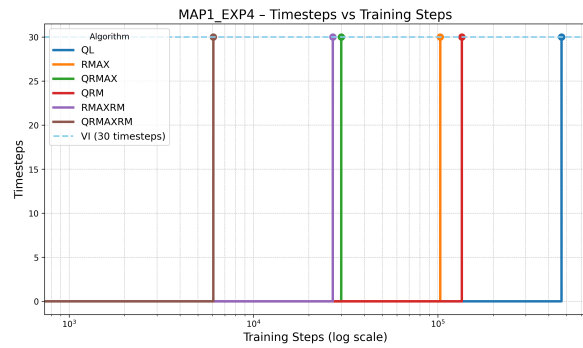


Figure 20: Map1 Exp4 - Office World

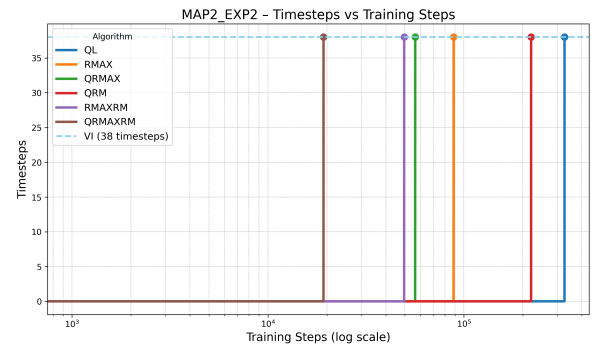


Figure 23: Map2 Exp2 - Office World

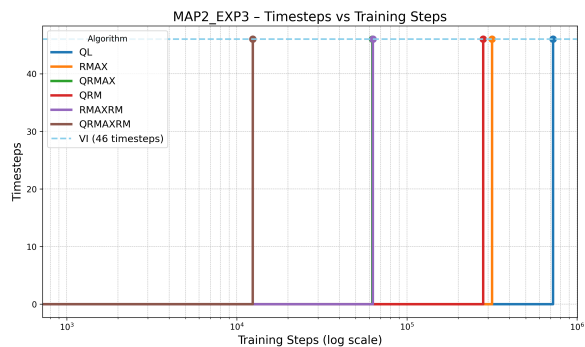


Figure 24: Map2 Exp3 - Office World

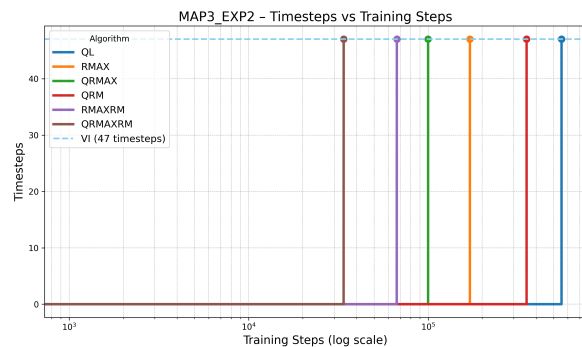


Figure 28: Map3 Exp2 - Office World



Figure 25: Map2 Exp4 - Office World

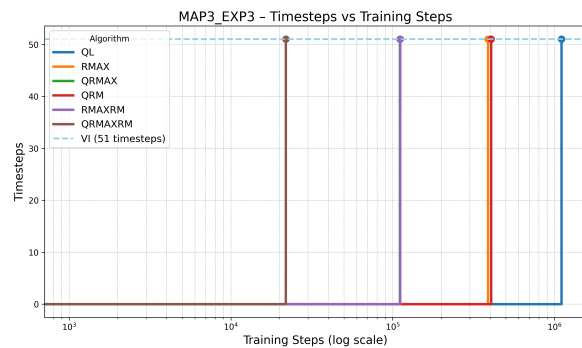


Figure 29: Map3 Exp3 - Office World

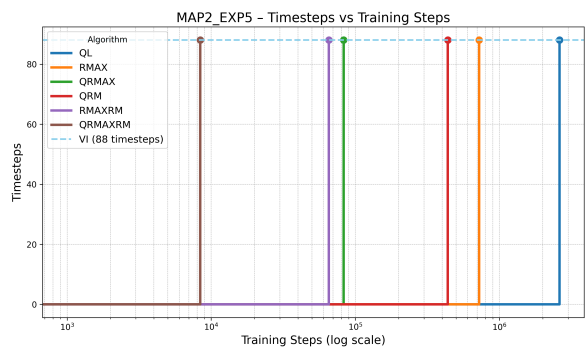


Figure 26: Map2 Exp5 - Office World

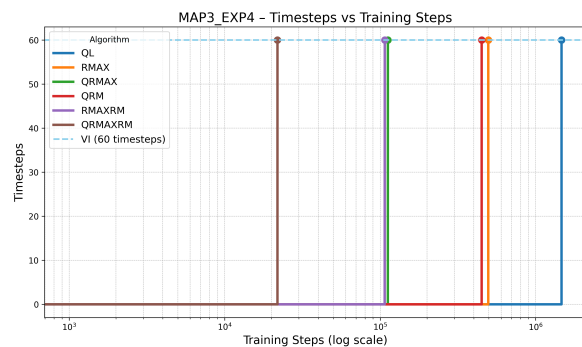


Figure 30: Map3 Exp4 - Office World

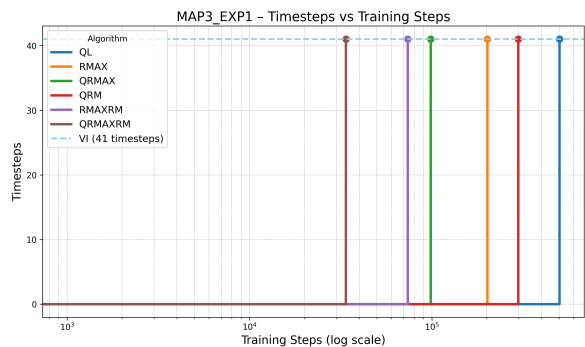


Figure 27: Map3 Exp1 - Office World

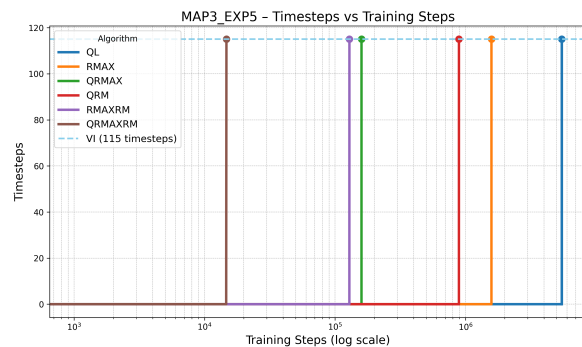


Figure 31: Map3 Exp5 - Office World